A CENTRALIZED AI-POWERED THESIS LIBRARY USING RETRIEVAL-AUGMENTED GENERATION

A Thesis

Presented to the Faculty of the

College of Computing Studies, Information and Communication Technology

Isabela State University

Echague, Isabela

In Partial Fulfillment of the

Academic Requirements for the Degree of

Bachelor of Science in Computer Science (BSCS)

Data Mining Track

By:

Ahron John F. Barlis

Carlo Rossi P. Gallardo

TABLE OF CONTENTS

TITLE PAGE **1**

TABLE OF CONTENTS **2**

CHAPTER I. INTRODUCTION **4**

1.1 Background of the Study 4

1.2 Objectives of the Study 5

1.3 Scope and Delimitation 7

1.4 Significance of the Study 11

1.5 Definition of Terms 12

CHAPTER II. THEORETICAL FRAMEWORK **16**

2.1 Review of Related Literature 16

2.1.1 Retrieval-Augmented Generation (RAG) Architecture 16

2.1.2 Large Language Models (LLMs) and Hallucination Mitigation 19

2.1.3 Traditional Library Systems and the Semantic Gap 22

2.1.4 Data Preprocessing and Information Overload 24

2.1.5 Educational RAG Systems and Traceable Citations (Related Studies) 25

2.1.6 System Evaluation, Privacy, and Security 26

2.1.7 Synthesis of the Literature 27

2.1.8 Gap and Relevance to the Study 29

2.2 Conceptual Framework 30

CHAPTER III. METHODOLOGY **35**

3.1 Materials 35

3.1.1 Software 35

3.1.2 Hardware 45

3.1.3 Data 46

3.2 Methods 47

3.2.1 Research Design 47

3.2.2 Software Development Life Cycle....52
3.2.3 System Procedures....55
3.2.4 System Evaluation....61
3.2.5 Statistical Treatment of Data....63
3.2.6 Ethical Considerations....66
3.3 System Architecture....68

CHAPTER IV. RESULTS AND DISCUSSION....71

4.1 Conceptualization and Architecture of the Localized RAG + LLM Knowledge Retrieval Model....71
4.2 Comparative Performance Analysis: Baseline LLM versus Proposed RAG + LLM....74
4.3 Application and Integration of the Centralized AI-Powered Thesis Library System....78
4.4 Internal Software Quality Evaluation under ISO/IEC 25010....81
4.4.1 Computational Observations....85

CHAPTER V. SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS....87

5.1 Summary of Findings....87
5.2 Conclusions....90
5.3 Recommendations....91

REFERENCES....93

CHAPTER I

INTRODUCTION

1.1 Background of the Study

At Isabela State University, specifically within the College of Computing Studies, Information and Communication Technology (CCSICT), students rely on manual navigation of digital soft copies and physical hardbound materials in the college library. Current traditional methods involve rigid keyword-based systems that depend solely on the abstract section of a study, failing to capture the full semantic context and technical depth of the research.

This reliance on manual browsing and abstract only searching leads to inefficient literature reviews and a high risk of unintentional topic duplication. While students may attempt to use general Large Language Models (LLMs) to improve synthesis, these public models lack access to CCSICT's private repository. Without access to localized data, standard LLMs are prone to hallucinations, generating false citations instead of providing verifiable local studies. Research in intelligent archiving (Custer & Sethi, 2017) highlights that static repositories often become "data graveyards" when they lack interactive retrieval mechanisms.

From a data mining perspective, extracting value from unstructured thesis manuscripts requires advanced text-mining methodologies. Classical data mining strictly relies on structured, tabular datasets, which fail to capture the semantic depth of academic research. However, this study utilizes embedding models to perform Unstructured Text Mining. By converting raw academic text into high dimensional mathematical vectors, the

system computationally mines semantic relationships and contextual proximity utilizing mathematical distance metrics like Cosine Similarity, which traditional keyword mining algorithms cannot achieve.

To address this gap, the study implements a Retrieval-Augmented Generation (RAG) architecture using the LangChain framework. The system utilizes PyMuPDF to extract text directly from digitized CCSICT theses, indexing the full text in a Supabase vector database. LangChain's orchestration then feeds relevant contextual data into a Large Language Model via the Gemini API. This closed-loop RAG implementation ensures that the AI generates responses grounded in verified institutional data (Lewis et al., 2020). By enabling semantic search across entire documents rather than relying solely on abstracts, the architecture also addresses the "Lost in the Middle" phenomenon (Liu et al., 2024), where critical information is often buried within lengthy documents.

The proposed Web-Based Intelligent Thesis Library functions as an indirect knowledge retrieval system, enhancing the efficiency of RRL writing and enabling instant topic validation by providing AI-synthesized, citation-backed responses rather than requiring students to manually browse individual thesis documents. Researchers and faculty will benefit from automated redundancy checks, ensuring that new studies build upon existing work rather than duplicating it. The system's processing speed will be monitored using LangSmith to trace latency, and its response accuracy will be evaluated using the Ragas framework (Es et al., 2024) to verify that the AI demonstrably minimizes hallucinations, thereby maintaining the integrity of all retrieved institutional data.

1.2 Objectives of the Study

General Objective

To develop and implement a Centralized AI-Powered Thesis Library using a Retrieval-Augmented Generation (RAG) AI architecture to solve the semantic gap and improve research retrieval performance in terms of time and management for the College of Computing Studies, Information and Communication Technology (CCSICT).

Specific Objectives

Specifically, the study aims to:

1. Conceptualize and architect a localized knowledge retrieval model combining Retrieval-Augmented Generation (RAG) with a Large Language Model (LLM) utilizing semantic chunking, high-dimensional vector embeddings, and similarity retrieval pipelines.

2. Compare the performance of the baseline model, or Standard LLM, versus the proposed model (RAG + LLM) strictly in terms of factual accuracy and hallucination mitigation using real institutional queries.

3. Apply the developed model (LLM + RAG) for the development of the Centralized AI-Powered Thesis Library System.

4. Evaluate the system's internal quality using the ISO/IEC 25010 software product quality standard through automated software testing tools based on the following criteria:

4.1 Functional Suitability

4.2 Performance Efficiency

4.3 Reliability

4.4 Maintainability

1.3 Scope and Delimitation

The primary focus of this study is the engineering and comparative evaluation of a knowledge retrieval model using Retrieval-Augmented Generation (RAG) for the College of Computing Studies, Information and Communication Technology (CCSICT) at Isabela State University, Echague Campus.

The technical and experimental scope of this research includes the following parameters:

- *Data Digitization:* The study will involve the conversion of physical hardbound copies and unstructured digital softcopies from the CCSICT archives into searchable PDF formats suitable for semantic indexing. Scanning physical manuscripts to PDF is an operational prerequisite carried out before ingestion: the system's digitization phase extracts and, where necessary, OCRs text from an uploaded PDF for semantic indexing, and does not itself perform scanning or emit a new searchable PDF. The ingestion parser accepts both traditional five-chapter thesis manuscripts and IMRAD (Introduction, Materials and Methods, Results and Discussion) manuscripts.

- *System Architecture:* The system will utilize the LangChain framework as the primary orchestration layer. Gemini will provide the embeddings model to process the documents, and Supabase will serve as the vector database for document chunking and semantic indexing. The Gemini API will provide the Large Language Model (LLM) for text synthesis.

- *Experimental Testing:* Execution of a quantitative comparative test to measure and evaluate retrieval performance specifically calculated in terms of factual accuracy and hallucination mitigation between a standard baseline LLM and the proposed RAG + LLM architecture.

- *System Integration and Evaluation:* Integration of the finalized RAG model into a web-based Thesis Library System, followed by a purely technical evaluation using the ISO/IEC 25010 software product quality standard focusing strictly on internal quality characteristics: Functional Suitability, Performance Efficiency, Reliability, and Maintainability, executed through automated testing tools.

To maintain technical feasibility, ensure controlled experimental variables, and uphold academic integrity, the researchers established the following delimitations:

- *Corpus Restrictions:* The vector database is strictly limited to theses generated within the CCSICT department, comprising both student (undergraduate) and faculty manuscripts across all CCSICT degree programs and specializations (BSCS, BSIT, BSIS, BLIS, and BSDSA), each labelled by category and filterable

in browsing, chat retrieval, and analytics. Research outputs from other colleges, external institutions, or other ISU campuses are entirely excluded from the dataset.

- *External Knowledge Isolation:* To actively prevent AI hallucinations and guarantee epistemic fidelity, the retrieval system is architecturally restricted from searching the open internet. It operates exclusively as a closed-domain retrieval assistant utilizing only local institutional data.

- *Content Generation Limits:* The system is explicitly programmed to retrieve and synthesize existing archived data. It will not generate original research content, automatically write thesis chapters, or construct independent academic arguments.

- *Network Dependency:* The proposed architecture requires a continuous, active internet connection to communicate with the cloud-based vector database and the LLM API. Consequently, offline functionality or local-network-only deployment is completely outside the parameters of this project.

- *Institutional Storage and Five-Year Recency Policy:* The repository indexes the CCSICT thesis collection across all departmental degree programs and specializations. To align with institutional research recency standards without losing archival history, the system enforces a dual-tier storage and recency policy: all digitized theses are permanently retained in the repository, while the Archive, RAG Chat, and Novelty Check interfaces provide a recency scope toggle between the complete archive (All theses) and the active five-year window (Last 5 years). Furthermore, the concurrent Batch Upload subsystem enables continuous ingestion as new cohorts complete their manuscripts each academic year.

- *Complex Data Extraction Limits:* Complex visual formats such as raw Entity-Relationship Diagrams (ERDs), unformatted code blocks, and unstructured image-based tables that fail layout-aware PDF parsing will not be semantically indexed to prevent vector space corruption. The system will primarily rely on the surrounding explanatory text for these visuals.

- *Indirect Access Model:* The system is designed as an indirect thesis library. Users will not have the capability to directly view, download, or browse the full-text content of any thesis document stored in the database. The system exclusively provides AI-generated responses that synthesize and cite relevant information from the archived theses. This design preserves the intellectual property of the original thesis authors while still enabling knowledge discovery.

- *Duplication Parameter Limit:* To prevent topic redundancy, the system evaluates new submissions against CCSICT archived data using an 85% cosine similarity threshold. If a proposed research query meets or exceeds this limit, the system flags it as a potential duplicate to encourage novelty. At this threshold, the integrated language model automatically alerts the user that the topic already exists and displays the exact similarity percentage. To provide immediate context for both the student and their faculty adviser, the model also generates a brief summary of the matched archival study. The system reports three complementary figures: the highest passage similarity found against the archive, the matched-chunk coverage (the percentage of the new manuscript's chunks whose nearest archived neighbour met the 85% threshold), and the single-paper similarity concentration (the percentage of the manuscript's chunks pointing to the single most similar archived

thesis). Single-paper concentration drives the deterministic advisory verdict of Clear (< 25%), Review Suggested (25% to < 70%), or High Overlap (≥ 70%) so that corpus growth across shared departmental templates does not falsely saturate the verdict. The verdict is advisory only: the system never automatically accepts or rejects a proposed topic, and the decision remains with the student and their faculty adviser. Users may scope novelty screening and chat retrieval to either the complete repository (All theses) or the active five-year institutional recency window (Last 5 years), and export a formal Microsoft Word Novelty Check Report bearing an official endorsement signature block for the Thesis Adviser and the CCSICT Dean.

1.4 Significance of the Study

The findings and outputs of this study will provide distinct advantages to the academic community of ISU Echague, particularly in modernizing local knowledge governance. The beneficiaries include:

Undergraduate Students. The proposed system will serve as an indirect retrieval assistant, reducing the time required to manually search through physical archives and unstructured digital files. Through semantic search capabilities, students will receive AI-synthesized responses that summarize relevant methodologies, related literature, and prior project scopes with traceable citations to the original thesis sources, without requiring direct access to complete thesis manuscripts. This improves the quality of literature reviews, minimizes unintentional topic duplication, and ensures access to reliable, locally sourced citations.

Faculty Members and Research Advisers. For instructors, the system provides a data-driven tool to validate the novelty of proposed student topics. Advisers can query the database to cross-reference new proposals against years of accumulated CCSICT theses, streamlining the title defense and approval process.

The CCSICT Department. The project establishes a structured, digitized knowledge base that preserves and protects the intellectual property of the college. By centralizing thesis data into a vector database accessible only through AI-mediated, indirect retrieval, the department transitions away from deteriorating physical copies and fragmented digital storage while ensuring that full thesis content is not directly exposed or downloadable, ensuring long-term institutional memory and compliance with modern data banking standards.

Future Researchers. Academically, this study contributes empirical data to the field of information retrieval. By executing a comparative factual accuracy test between a baseline LLM and a RAG-augmented LLM, future computer science researchers can utilize the results of this experiment as a technical baseline for scaling localized AI search engines in other academic settings.

1.5 Definition of Terms

To ensure clarity and precision, the following terms are defined based on how they are used within the context of this study. These terms are categorically organized in logical progression, prioritizing the core system architecture down to the evaluation metrics:

Centralized AI-Powered Thesis Library System. This refers to the final software application developed in this study. It is a web-based, AI-powered research assistant deployed for the CCSICT department that provides indirect access to CCSICT student and faculty thesis knowledge through semantically grounded, citation-backed AI responses rather than direct document browsing or downloading.

Retrieval-Augmented Generation (RAG). The core computational architecture proposed in this research algorithmically restricts the AI synthesis engine to generate answers based exclusively on retrieved vector embeddings from the localized CCSICT thesis archives, thereby ensuring locally cited outputs.

LangChain. The primary open-source orchestration framework utilized in the system backend to programmatically link the user's search query, the Supabase vector database, and the Gemini LLM into a unified data retrieval pipeline.

Unstructured Text Mining. The algorithmic process of extracting high-value semantic data and hidden patterns from unformatted text documents (PDFs) through natural language processing and vectorization.

Document Chunking. The programmatic process of dividing digitized CCSICT thesis manuscripts into smaller, discrete text blocks utilizing LangChain to enable precise semantic indexing within the vector database.

Vector Database. The cloud infrastructure, specifically Supabase with the pgvector extension, engineered to store the high-dimensional mathematical representations (embeddings) of the digitized CCSICT thesis corpus, allowing for rapid similarity searches.

Semantic Gap. The limitation observed in traditional library catalog systems, such as OPAC, that rely strictly on literal keyword matching, which frequently fails to interpret the conceptual and contextual intent behind a student's research query.

Semantic Search. The data retrieval technique that locates institutional information based on the mathematical proximity of their contextual meaning within a vector space, rather than relying on exact word matches.

Cosine Similarity. The specific mathematical metric used by the Supabase pgvector database to calculate the geometric distance and semantic relevance between the user's research query vector and the embedded document vectors.

Baseline Model. The standalone Large Language Model, specifically the standard Gemini API utilized as the control variable. It is evaluated without access to the local database to scientifically establish an empirical baseline of hallucination severity to benchmark factual accuracy against the proposed RAG architecture.

AI Hallucination. In this study, this refers to the generative error where an unaugmented Large Language Model fabricates academic citations, methodologies, or data when queried about ISU Echague's institutional research, producing convincing but mathematically unverified information.

ISO/IEC 25010. The international software quality framework used by the researchers to quantitatively assess the final web system, specifically measuring its internal quality characteristics Functional Suitability, Performance Efficiency, Reliability, and

Maintainability through automated software testing and static code analysis rather than subjective user evaluation.

Duplicate Parameter. The predefined mathematical threshold set at 85% cosine similarity. It serves as the algorithmic trigger that allows the system to identify and flag a student's research query as highly redundant compared to an existing CCSICT study, ensuring new research builds upon rather than copies existing work.

CHAPTER II

THEORITICAL FRAMEWORK

2.1 Review of Related Literature

2.1.1 Retrieval-Augmented Generation (RAG) Architecture

To bridge the semantic gap of traditional systems, the computational linguistics community has converged on the Retrieval-Augmented Generation (RAG) architecture. RAG structurally separates an AI's linguistic reasoning capabilities from its knowledge storage, relying on external retrieval rather than internal memory. Lewis et al. (2020) introduced the original RAG framework, which connects a language model to a searchable index of documents. Their tests across several tasks showed that RAG produces more accurate answers than a language model working from memory alone, ensuring that generated responses are strictly grounded in verified databases.

Khaliq et al. (2024) studied the optimization of academic queries using retrieval-augmented large language models. They explicitly used the LangChain framework, recursive character text splitters, and vector databases, finding that text-splitting chunk sizes of 700 to 800 tokens optimize LLM responses for academic datasets. Mezhlumyan developed a RAG system for academic papers using a two-layer vector search. The project detailed how to bypass token limits when querying long academic PDFs by utilizing an HNSW index and SBERT to first find relevant thesis abstracts and then narrow down to specific paragraphs (Mezhlumyan, 2024).

Arivazhagan et al. (2023) argued that dense retrievers can generalize poorly out-of-domain and proposed a hybrid hierarchical retrieval approach. Their approach combined sparse and dense retrievers across stages, improving robustness for specialized vocabulary and varying user queries. Supabase documented the use of pgvector for embeddings and vector similarity search. Their technical documentation provided implementation guidance for combining a library repository with a vector search backend, demonstrating the foundational data architecture required for such deployments (Supabase, 2026). These studies show that a robust RAG architecture requires careful orchestration of document loaders, semantic chunking, and multi-layered vector searches to process academic literature efficiently.

Algorithmic Flow of Retrieval-Augmented Generation (RAG)

The RAG architecture operates through a bipartite algorithmic process: the Retrieval Phase and the Generation Phase.

In the Retrieval Phase, a user's natural language query is first processed by an embedding model, specifically Gemini Embedding (models/gemini-embedding-001). Unlike traditional keyword tagging, this process falls under the domain of **Unstructured Text Mining**. The model performs deep feature extraction, mapping the semantic meaning of the text into a high-dimensional continuous vector space, typically consisting of 768 dimensions.

Once vectorized, the algorithm executes a nearest-neighbor search against the Supabase pgvector database, which contains the pre-embedded chunks of institutional thesis documents. The retrieval is governed by **Cosine Similarity**, a mathematical distance

metric that calculates the cosine of the angle between the query vector (A) and the document vector (B). Mathematically expressed as

$$Cosine\ Similarity = \frac{A \cdot B}{||\ A\ ||||\ B\ ||}$$

Where:

A = Query vector (the user's research question converted into a high-dimensional embedding)

B = Document vector (a pre-embedded thesis chunk stored in the Supabase pgvector database)

$A \cdot B$ = Dot product of vectors A and B

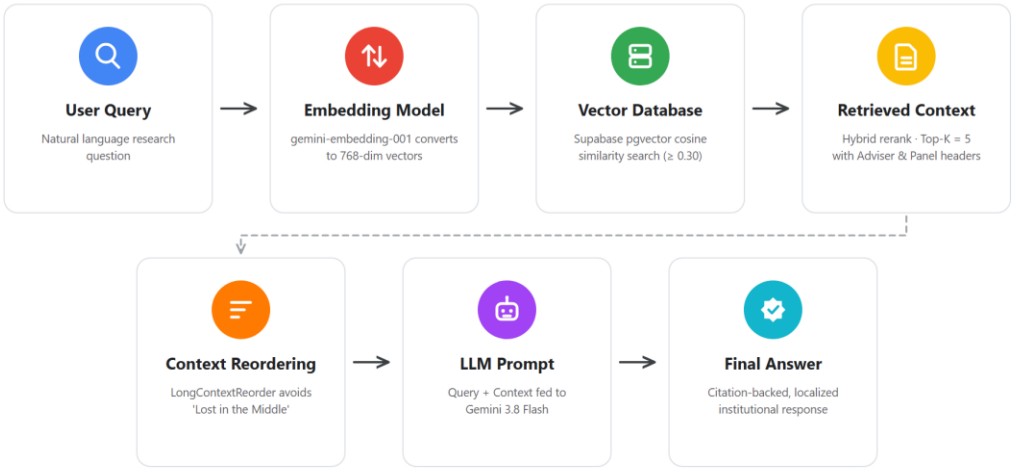
$||\ A\ ||||\ B\ ||$ = Product of the magnitudes of vectors A and B

This formula ensures that documents sharing the closest contextual meaning even if they do not share exact keywords are pulled from the database.

In the Generation Phase, these retrieved top-k document chunks are concatenated with the user's original query to form an augmented prompt. Instead of relying on its parametric memory, which risks hallucination, the LLM algorithm utilizes its self-attention mechanisms to synthesize an answer strictly derived from the appended context window. This algorithmic pipeline mathematically grounds the generated text in the localized data, ensuring verifiability (see Figure 1).

Figure 1

Algorithmic Flow of Retrieval-Augmented Generation (RAG)



2.1.2 Large Language Models (LLMs) and Hallucination Mitigation

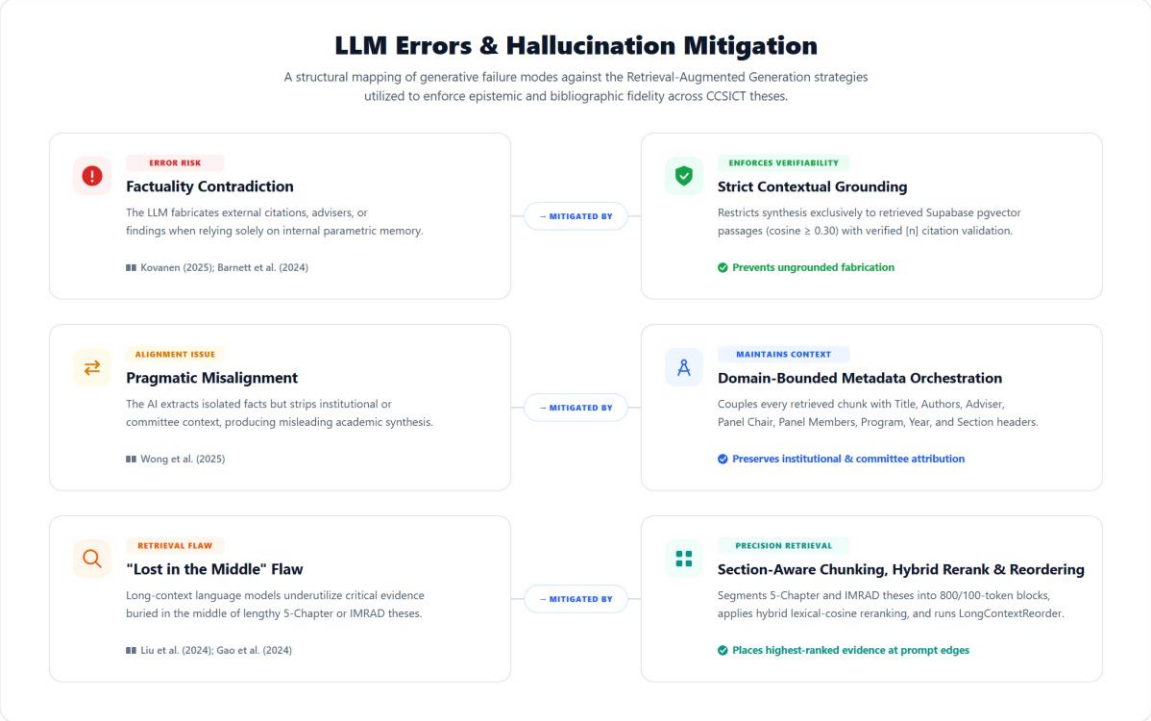
While general LLMs possess powerful natural language understanding, their direct application in academic research introduces profound risks of hallucinations, making targeted context retrieval and faithfulness critical. Kovanen (2025) explored the taxonomy of AI errors by distinguishing between factuality contradictions and faithful hallucinations. The study defined faithfulness as the AI's ability to stick strictly to the uploaded context without fabricating outside information. Barnett et al. (2024) identified seven critical failure points when engineering RAG systems, including incomplete retrieval, missed context, and wrong-format outputs, demonstrating the strict engineering constraints required to balance factual grounding and answer coverage. Their findings provide a practical engineering checklist for diagnosing and preventing retrieval failures in production RAG deployments.

Gao et al. (2024) surveyed a wide range of RAG architectures and noted that knowledge representation strategies particularly how text is semantically chunked significantly influence LLM accuracy and hallucination rates. Their survey synthesized findings from multiple studies demonstrating that semantically chunking texts such as separating methodology from results yields higher AI accuracy and drastically reduces hallucinations. Wong et al. (2025) warned that RAG systems can suffer from pragmatic misalignment. Their critical paper explained that systems might pull technically accurate facts but take them out of context, leading to misleading answers and becoming dangerous communicators if not properly bound.

Liu et al. (2024) showed that long-context language models underutilize information located in the middle of long prompts. Their study on the "Lost in the Middle" phenomenon strongly supported the need for careful chunking, retrieval ordering, and semantic search to pinpoint relevant sections rather than stuffing chapters into prompts. Gao et al. (2024) similarly addressed this phenomenon, noting how information buried in hundreds of pages requires targeted retrieval to maintain model effectiveness. These studies show that while RAG mitigates basic hallucinations, ensuring true faithfulness requires advanced semantic chunking, context reordering, and an awareness of pragmatic misalignment to guarantee accurate academic synthesis (see Figure 2).

Figure 2

Identified LLM Error Behaviors and Corresponding RAG Mitigation Strategies



2.1.3 Traditional Library Systems and the Semantic Gap

Traditional library systems, such as Online Public Access Catalogues (OPAC), are foundational to academic institutions but often struggle with the semantic complexity of modern research. These systems typically operate on rigid Boolean logic and literal keyword matching, which limits their ability to process natural language queries effectively. Bevara et al. (2025) explored the shift from traditional keyword-based Boolean library searches to Natural Language Retrieval-Augmented Generation (RAG) systems. Their work discussed metadata indexing and real-time query processing, proving that traditional systems like OPAC fail at handling complex student queries. The authors found that this failure creates a strict demand for semantic AI search capabilities.

Xuan and Shaw (2025) presented a real-world case study of an academic library implementing RAG to allow users to search their institutional repository using natural language instead of rigid metadata. Their study showed that top-tier institutions globally are replacing traditional OPAC systems with RAG, indicating that this modernization is a necessary technological transition for academic archives. These studies show that traditional library systems operating strictly on literal word matching cannot interpret the semantic intent behind complex academic searches, leading to high rates of unintentional topic duplication and the necessity of semantic AI modernization.

Local Context

Within the local context of the Philippines, the limitations of traditional library systems are heavily documented. A study by Doliente et al. (2023) conducted at a higher educational institution in Mindanao revealed that the majority of students rarely utilize the traditional Online Public Access Catalog (OPAC) due to user dissatisfaction, rigid search functionalities, and an inability to interpret complex research queries. Additionally, Armilla and Abangan (2025) assessed the digitization of academic libraries in the Philippines, highlighting that while State Universities are transitioning from physical hardbound copies to digital PDFs, they face massive technological bottlenecks in information retrieval.

To resolve this, Philippine academic sectors are beginning to explore AI. Sambrano et al. (2025) studied state university students in Region I and found a massive shift in library patronage, noting that students now prefer AI tools over traditional OPACs strictly because of "faster information retrieval" and natural language understanding. Even at the

national level, the Supreme Court of the Philippines Library recently initiated the redevelopment of their E-Library to transition away from keyword-matching, integrating AI natural language processing to allow users to search legal precedents using conversational queries (Supreme Court of the Philippines, 2024). These local studies validate the urgent necessity for State Universities like ISU Echague to modernize their unstructured thesis archives using semantic AI architectures.

2.1.4 Data Preprocessing and Information Overload

The inability of traditional systems to facilitate deep, interactive querying directly leads to the manifestation of the "data graveyard" phenomenon, where unstructured academic data becomes buried and unusable. Custer and Sethi (2017) highlighted that static repositories often become "data graveyards" because they lack interactive retrieval mechanisms. Their research pointed out that vast amounts of collected institutional data are stored but rarely accessed or interpreted due to severe usability hurdles.

Vercruysse et al. (2024) focused on extracting semantic graphs and structuring Linked Data into plain objects for developers. Their study highlighted the complexity of raw data extraction and supported the argument that converting unstructured academic data, such as digitized PDFs, into structured, readable formats requires specialized semantic lenses. Gokdemir et al. (2025) detailed a High-Performance Computing (HPC) RAG workflow that parsed over 3.6 million scientific PDFs. They introduced a layout-aware PDF parser called Oreo and a query-aware encoder, explaining the massive computational challenges of extracting text from hardbound scanned theses and defending the need for dedicated data preprocessing stages.

Aytar et al. (2025) focused on using GROBID for data cleaning, semantic chunking, and an abstract-first retrieval method. Their framework helped navigate literature without information overload by quickly scanning thesis abstracts before performing a deep dive into the full PDF content, saving significant computational power. These studies show that unstructured academic texts pose a significant challenge, and that without proper layout-aware parsing and abstract-first retrieval strategies, documents risk becoming completely inaccessible data graveyards.

2.1.5 Educational RAG Systems and Traceable Citations (Related Studies)

The application of RAG within educational domains provides institutional support and enforces academic verifiability through explicitly traceable citations. Li et al. (2025) reviewed dozens of studies applying RAG across educational scenarios through a systematic survey. The survey identified recurring challenges such as hallucination and evaluation gaps, strengthening the argument that RAG is an established and highly necessary approach for educational support.

Oreški and Vlahek (2024) described the development of an AI chatbot for student support based on RAG. Their case study reported prototyping choices motivated by domain knowledge gaps, illustrating how RAG can be operationalized as an academic assistant strictly grounded in institutional content. Besrour et al. (2025) detailed a system providing fine-grained in-line citations for scientific question-answering. Their multi-agent RAG system allowed users to trace claims back to specific sentences, validating that RAG can eliminate hallucinations by providing explicit, verifiable citations from local archives.

Gao et al. (2023) introduced the ALCE benchmark for generating text with citations. They argued that citations improve verifiability and constrain hallucinations, demonstrating that responses must treat citations as a first-class output to support academic claims effectively. These studies show that educational RAG applications significantly benefit students by acting as domain-bounded assistants, and that enforcing explicit in-line citations is critical for maintaining academic integrity and verifiability.

2.1.6 System Evaluation, Privacy, and Security

The successful deployment of a Web-Based Intelligent Thesis Library necessitates rigorous, quantifiable evaluation methodologies and strict security protocols to protect institutional intellectual property. Santra et al. (2025) discussed building a locally deployed conversational search tool called the BiblioGPT prototype. Their work emphasized protecting user privacy and institutional data by avoiding the risks of public cloud LLMs, proving that limiting AI to a local database through "Knowledge Isolation" is a secure practice.

Bodea et al. (2026) created a taxonomy of privacy risks in RAG systems. Their study detailed data leakage, prompt leakage, and database leakage, explaining how processing institutional data locally mitigates these vulnerabilities. The Open Web Application Security Project (OWASP) listed major security risks for LLM applications, including prompt injection and sensitive data disclosure. Their standards provide authoritative justification for strict access controls and data separation in RAG pipelines (OWASP, 2025).

To further mitigate institutional data exposure, the system will utilize the Gemini API under deployment terms that prohibit the retention or use of submitted data for model training, ensuring compliance with Google's API data governance policy (Google, 2024). This guarantee is bounded by the provider actually serving each call: the architecture permits generation to be routed through an OpenAI-compatible gateway without changing the model, and a third-party operator is not covered by those terms. The direct Google route will therefore be used for the formal evaluation and for any processing of the governed corpus, and the release fingerprint will record which route produced each result. Embedding calls are never routed and reach Google in every configuration. Brown et al. (2025) published a highly comprehensive review of 128 RAG studies that categorized evaluation metrics. Their systematic literature review justified the use of precision, recall, F1-score, and LLM-as-a-judge methodologies for evaluating RAG architectures objectively. Es et al. (2024) introduced the RAGAs automated evaluation framework. Their system demo emphasized evaluating both retrieval and generation modules using reference-free metrics without requiring massive human annotation. Ru et al. (2024) proposed RAGChecker, a fine-grained framework for diagnosing RAG. Their work highlighted claim-level evaluation, helping to separate errors caused by the retriever missing evidence from errors caused by the model hallucinating despite having the correct chunk. These studies show that deploying a thesis library requires strict local knowledge isolation to prevent data leakage, paired with modular automated frameworks to continuously evaluate retrieval precision and generation faithfulness.

2.1.7 Synthesis of the Literature

The reviewed literature converges on several critical findings that collectively establish the theoretical and technical foundation for this study. Across the domains of RAG architecture, LLM hallucination mitigation, traditional library systems, data preprocessing, educational applications, and evaluation frameworks, a consistent pattern emerges: existing academic retrieval systems are fundamentally limited by the absence of an integrated, semantically intelligent pipeline.

First, the studies on RAG architecture (Lewis et al., 2020; Khaliq et al., 2024; Mezhlumyan, 2024; Arivazhagan et al., 2023) collectively establish that connecting language models to external document indexes through vector-based retrieval produces significantly more accurate and verifiable outputs than relying on parametric memory alone. These findings are directly reinforced by the hallucination mitigation literature (Kovanen, 2025; Barnett et al., 2024; Wong et al., 2025), which identifies the specific failure modes including fabricated citations, pragmatic misalignment, and the "Lost in the Middle" phenomenon (Liu et al., 2024; Gao et al., 2024) that occur when language models operate without grounded retrieval. Together, these two bodies of research demonstrate that RAG is not merely an enhancement but a necessary architectural constraint for any AI system intended to handle academic queries with factual integrity.

Second, the literature on traditional library systems (Bevara et al., 2025; Xuan & Shaw, 2025; Doliente et al., 2023; Sambrano et al., 2025) consistently documents the failure of keyword-based OPAC systems to interpret the semantic intent behind complex research queries. This limitation directly connects to the data preprocessing literature

(Custer & Sethi, 2017; Gokdemir et al., 2025; Aytar et al., 2025), which explains that without layout-aware parsing and semantic structuring, institutional archives become inaccessible "data graveyards." The convergence between these findings establishes a cause-and-effect relationship: rigid keyword systems create the access barrier, while unprocessed data worsens it.

Third, the educational RAG literature (Li et al., 2025; Oreški & Vlahek, 2024; Besrour et al., 2025; Gao et al., 2023) demonstrates that RAG successfully serves as a domain-bounded academic assistant when paired with traceable citations. However, the evaluation and security literature (Brown et al., 2025; Es et al., 2024; Ru et al., 2024; Santra et al., 2025; Bodea et al., 2026; OWASP, 2025) cautions that deployment must include strict local knowledge isolation and automated evaluation frameworks to prevent data leakage and ensure continuous generation faithfulness.

Despite these well-documented findings, the reviewed studies address these challenges largely in isolation. RAG studies focus on general question-answering benchmarks rather than localized institutional theses. Hallucination studies examine model-level errors but rarely contextualize them within academic library retrieval. Traditional library studies identify the semantic gap but stop short of implementing AI solutions. Educational RAG studies operate on curated datasets rather than raw, unstructured, digitized thesis manuscripts. No single study integrates all these components: document digitization, semantic chunking, vector-based retrieval, LLM synthesis with traceable citations, hallucination mitigation, and automated evaluation into a unified, closed-loop pipeline specifically tailored for an institutional thesis archive.

2.1.8 Gap and Relevance to the Study

This is the specific gap that the present study addresses. Existing university library systems rarely integrate a complete, closed-loop RAG pipeline specifically designed to process digitized academic theses using vector databases for semantic indexing and LLMs for citation-backed synthesis. Most commercial LLMs suffer from ungrounded hallucinations because they lack access to local institutional databases, while traditional OPACs lack the natural language understanding needed for complex academic discovery. The proposed system bridges this gap by utilizing PyMuPDF to extract text directly from digitized CCSICT theses, a Supabase vector database for semantic indexing, and an LLM constrained entirely to this localized context. By combining these technologies, the system will provide faster RRL writing, easy topic validation, automated redundancy checking, and will statistically minimize AI hallucinations regarding local references, thereby cultivating a high-standard research culture.

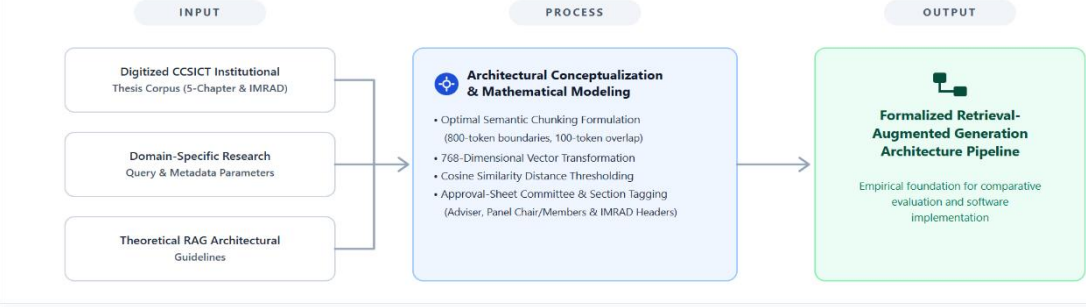
From a Data Mining perspective, the vectorization of unstructured thesis PDFs constitutes a form of unsupervised feature extraction, a core data mining technique. The subsequent nearest-neighbor retrieval using Cosine Similarity mirrors the classification and clustering paradigms foundational to data mining methodology. This positions the study firmly within the Data Mining track by applying mathematical text-mining operations to extract actionable knowledge from previously inaccessible institutional archives.

2.2 Conceptual Framework

The conceptual framework of this study is structured into four distinct Input-Process-Output (IPO) models, directly mapping to the study's specific objectives. This modular approach outlines the logical progression from theoretical conceptualization to final system evaluation.

Figure 3

Objective 1: Foundational Architecture



Input: The *Digitized CCSICT Institutional Thesis Corpus*, *Domain-Specific Research Query Parameters*, and *Theoretical RAG Architectural Guidelines* provide the foundational parameters required to conceptualize the localized retrieval architecture.

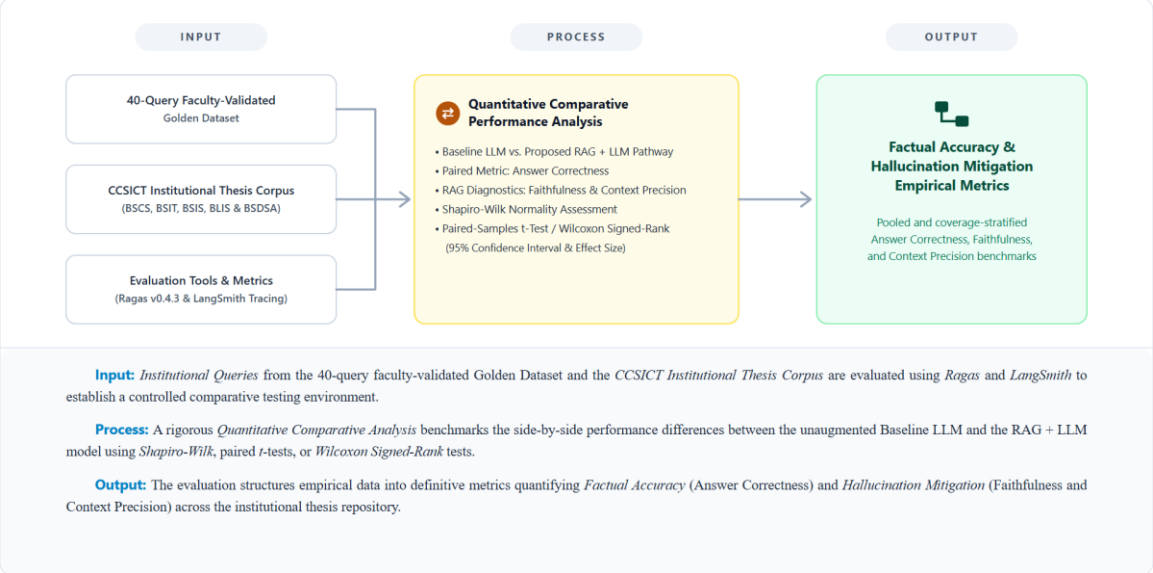
Process: *Architectural Conceptualization* and *Mathematical Modeling* formulate the optimal semantic chunking boundaries (800-token chunks with 100-token overlap), 768-dimensional vector transformation, and cosine similarity distance thresholding.

Output: The phase yields the *Formalized Retrieval-Augmented Generation Architecture Pipeline*, serving as the empirical foundation for comparative performance evaluation and full-stack software implementation.

As illustrated in Figure 3, the Input consists of the Digitized CCSICT Institutional Thesis Corpus, Domain-Specific Research Query Parameters, and Theoretical RAG Architectural Guidelines. The Process involves Architectural Conceptualization and Mathematical Modeling, comprising optimal semantic chunking formulation (800-token boundaries with 100-token overlap), 768-dimensional vector transformation, and cosine similarity distance thresholding. The Output yields the Formalized Retrieval-Augmented Generation Architecture Pipeline, serving as the empirical foundation for comparative evaluation and software implementation.

Figure 4

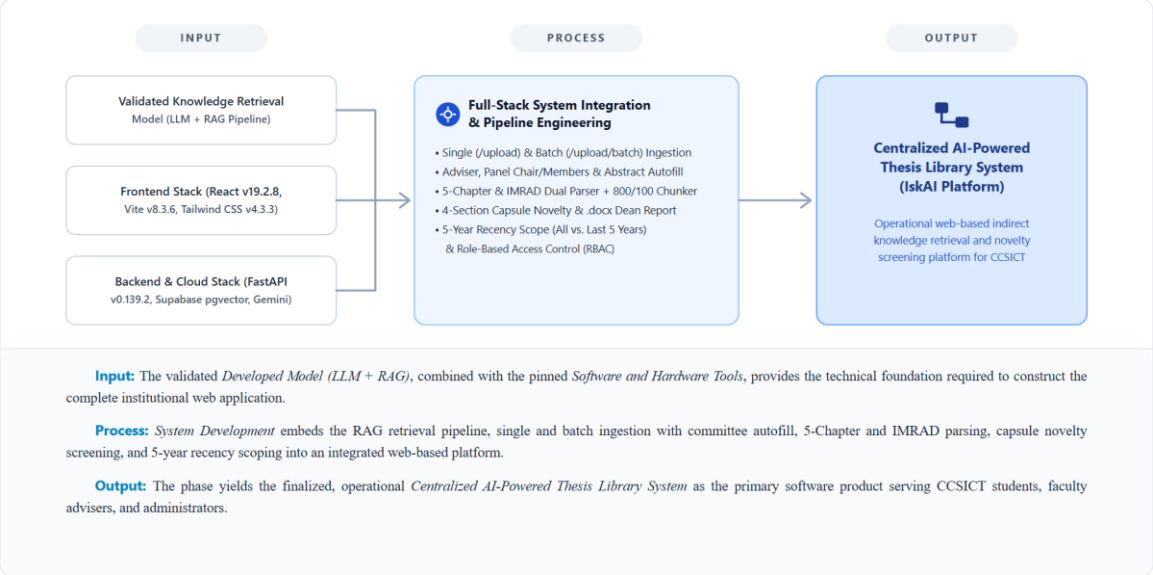
Objective 2: Comparative Performance Analysis



As illustrated in Figure 4, the Input utilizes *Institutional Queries* representing the primary real-world data, which are assessed using established *Evaluation Tools & Metrics* to create a structured comparative testing environment. The Process involves a rigorous *Comparative Analysis* following a logical progression of scientific testing, evaluating the side-by-side performance differences between the baseline and the newly integrated model. The Output generated from this phase structures the empirical data into definitive metrics, specifically quantifying the models' capabilities in extending *Factual Accuracy* and enabling *Hallucination Mitigation*.

Figure 5

Objective 3: System Integration



Input: The validated *Developed Model (LLM + RAG)*, combined with the pinned *Software and Hardware Tools*, provides the technical foundation required to construct the complete institutional web application.

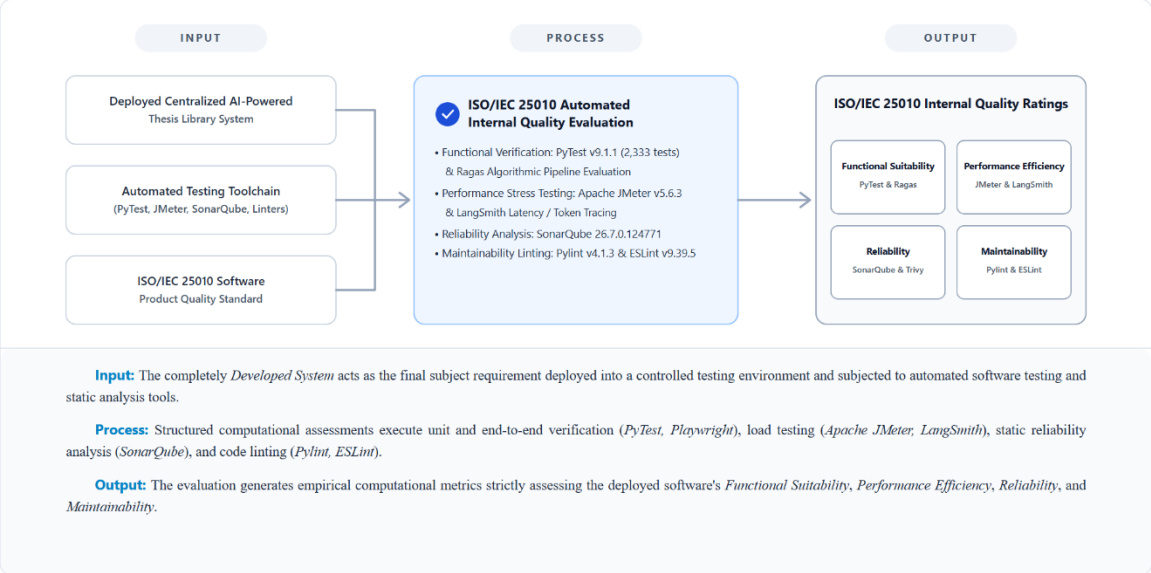
Process: *System Development* embeds the RAG retrieval pipeline, single and batch ingestion with committee autofill, 5-Chapter and IMRAD parsing, capsule novelty screening, and 5-year recency scoping into an integrated web-based platform.

Output: The phase yields the finalized, operational *Centralized AI-Powered Thesis Library System* as the primary software product serving CCSICT students, faculty advisers, and administrators.

As illustrated in Figure 5, the Input takes the validated Developed Model (LLM + RAG), combined with requisite Software & Hardware Tools, to provide the technical fundamentals needed to construct the complete software application. The Process of System Development acts as the core full-stack engineering stage. This involves embedding the optimized artificial intelligence framework into a tangible, integrated web-based platform. The Output yields the finalized, operational Centralized AI-Powered Thesis Library System as the primary tangible product of the study.

Figure 6

Objective 4: Formal Quality Evaluation



As illustrated in Figure 6, the Input for the final phase is the completely *Developed System*, which acts as the final subject requirement deployed into a controlled testing environment and subjected to automated software testing tools. The Process involves structured computational assessment procedures conducted following the standardized guidelines specified by the *ISO/IEC 25010* software product quality standard. Finally, the Output definitively generates empirical computational metrics, strictly evaluating the deployed software's *Functional Suitability*, *Performance Efficiency*, *Reliability*, and *Maintainability* through automated software testing tools.

CHAPTER III

METHODOLOGY

3.1 Materials

3.1.1 Software

For the programming and deployment phases, the researchers will utilize a specific stack of development tools and cloud services. Visual Studio Code will serve as the primary integrated development environment (IDE) used to write, edit, and organize the system's source code.

Table 1

Frontend Development Stack

Technology	Version	Purpose
React	v19.2.8	Will serve as the primary library for developing a component-based, highly responsive user interface for student researchers.
JavaScript (JSX)	ES6+	Will be utilized as the core scripting language for implementing front-end logic and dynamic rendering.

Vite	v8.3.6	Will function as the build tool and development server to ensure rapid hot-module replacement and optimized production bundling
Node.js	v24.18.0	Will provide the JavaScript runtime for the build toolchain, the development server, and the automated test runner.
Tailwind CSS	v4.3.3	Will supply the utility-first styling layer applied consistently across all interface views.
React Router	v8.3.0	Will handle client-side routing between the landing, archive, chat, and administrative views.
TanStack Query	v5.101.4	Will manage server state, caching, and background refetching for API-backed views.
Axios	v1.21.0	Will serve as the HTTP client for all requests from the frontend to the FastAPI backend.
Framer Motion	v12.42.2	Will provide the animation and transition primitives used by the interface components.

@supabase/supabase-js	v2.110.8	Will provide the browser client for Supabase authentication, session refresh and multi-factor enrolment, on which the sign-in, sign-up and password-reset flows depend.

Table 2

Backend Software Specifications

Technology	Version	Purpose
Python	v3.14.7	Will serve as the backend runtime, pinned to an exact patch level and asserted at container build time.
FastAPI	v0.139.2	Will be employed as the web framework to build a high-performance REST API capable of handling asynchronous requests.
Pydantic	v2.13.4	Will be utilized for strict data validation and settings management through Python type hinting.

python-multipart	v0.0.32	Will be integrated to facilitate the parsing of multipart form data, specifically for handling document uploads.
python-dotenv	v1.2.2	Will manage environment variables to maintain secure configuration of API keys and database credentials.
pydantic-settings	v2.14.2	Will load and validate server configuration from environment variables at startup.
Uvicorn	v0.51.0	Will run the ASGI server hosting the FastAPI application.
SlowAPI	v0.1.10	Will enforce per-route and default rate limits that protect the API from abuse and runaway cost.
redis (Python client)	v8.0.1	Will connect the API to the Redis server (redis 8.6.4 in the operations compose file) that backs rate limiting and the shared daily guest token allowance.

cryptography	v50.0.0	Will encrypt the archival storage backups using AES-GCM encryption under a scrypt-derived key; pinned at v50.0.0 to clear CVE-2026-69247.
PyJWT	v2.15.1	Will verify and read the Supabase-issued JSON Web Tokens: the assurance level behind the privileged multi-factor check and the user identity behind per-user rate limits. Tokens are issued by Supabase Auth, never by the backend.
HTTPX	v0.28.1	Will perform outbound HTTP calls to Supabase and the Gemini API.

Table 3

AI and RAG Orchestration Tools

Technology	Version	Purpose
Supabase	v2.31.0	Will be utilized as the client interface for connecting the back-end to the PostgreSQL-based cloud database.

pgvector	v0.7.0 (Managed via Supabase)	Will be enabled within the database to support vector similarity searches for semantic document retrieval.
langchain-google-genai	v4.3.1	Will facilitate the secure integration of Google's Generative AI models into the system's backend.
Gemini 3.8 Flash	gemini-3.8-flash	Will be employed as the primary Large Language Model (LLM) for synthesizing grounded, citation-backed responses and front-matter metadata extraction.
Gemini Embedding	models/gemini-embedding-001 (768 dimensions)	Will be utilized to generate high-dimensional vector embeddings for indexing and searching document contents.
Gemini 3.8 Flash (Verdict & Judge)	gemini-3.8-flash	Will write the advisory verdict summaries and follow-up answers of novelty scans and serve as the Ragas automated evaluator judge in the Objective 2 comparison.

langchain-core	v1.5.1	Will provide the Runnable and prompt abstractions that compose the retrieval-to-generation chain.
langchain-text-splitters	v1.1.2	Will supply the RecursiveCharacterTextSplitter used to chunk extracted manuscript text.
langchain-openai	v1.4.1	Will provide the OpenAI-compatible chat client used only when generation is routed through a gateway; embeddings are never routed through it.
tiktoken	v0.13.0	Will provide the cl100k_base tokenizer used as a fixed proxy for measuring chunk sizes.
PyMuPDF	v1.28.0	Will extract text and layout information directly from uploaded PDF manuscripts.

tesserocr	v2.10.0	Will bind the Tesseract OCR engine for image-based scanned pages.
Pillow	v13.0.0	Will handle page image preparation on the OCR path.
LangSmith	v0.10.10	Will trace and log pipeline latency and token consumption during evaluation runs.

Table 4

Software Testing and Quality Assurance Tools

Technology	Version	Purpose
PyTest	v9.1.1	Will be employed as the automated testing framework for executing unit tests and integration tests to verify the

		functional correctness of API endpoints and RAG pipeline operations.
Apache JMeter	v5.6.3	Will be utilized as a load-testing tool to simulate concurrent user requests and measure API response times, throughput, and system performance under stress conditions.
SonarQube	Community Build 26.7.0.124771	Will be deployed as a continuous static code analysis platform to detect code smells, logic flaws, unhandled exceptions, and potential reliability vulnerabilities in the backend codebase.
Pylint	v4.1.3	Selected to generate automated, objective code quality scores by evaluating the structural maintainability, modularity, and strict typing compliance of the Python backend.
ESLint	v9.39.5	Selected to enforce strict ECMAScript standards and evaluate the structural maintainability and technical debt

		mitigation of the React frontend codebase.
Node.js test runner	v24.18.0 (node --test)	Will execute the frontend unit test suite using the runtime's built-in test runner.
Playwright	v1.61.1	Will drive end-to-end browser tests across the critical user journeys of the interface.
axe-core for Playwright	v4.12.1	Will perform automated accessibility checks within the end-to-end suite.
SonarScanner CLI	v8.0.1.6346	Will submit the analysis to the SonarQube server that produces the retained Reliability evidence.
Trivy	trivy-action v0.36.0	Will scan both container images on every push and fail the pipeline on a CRITICAL or HIGH vulnerability that already has a fix available.

Gitleaks	gitleaks-action v2	Will scan the full commit history for committed credentials on every push and pull request.
pip-audit	installed at run time	Will audit the pinned Python dependency lock against the Python Packaging Advisory Database.
npm audit	bundled with Node.js v24.18.0	Will audit the production JavaScript dependency tree and fail on a high-severity advisory.

3.1.2 Hardware

To develop and test the system effectively, the researchers will use a laptop with specific hardware parts. Table 5 specifies the development workstation rather than a deployment requirement: all embedding and generation work is performed remotely through the Gemini API and OCR is CPU-bound, so the discrete GPU is not used by the architecture. These components are chosen to make sure the software runs smoothly and maintains a stable connection to the cloud-based AI services. The table below lists the primary hardware used in this study. Its Gap column records the researchers' readiness

against each requirement: a component marked Feasible is already available on the development workstation, so nothing has to be procured before implementation begins.

Table 5

Hardware Specifications

Hardware Component	Description & Specifications	Gap
CPU	Intel Core i5 13th Gen	Feasible
GPU	Nvidia RTX 4050	Feasible
RAM	16GB 4800MHz	Feasible
Storage	1TB M.2 Gen 4	Feasible
Network Adapter	Wi-Fi 6 Wireless capability	Feasible
Network High-speed Broadband Internet	Required for continuous real-time communication with the Supabase vector database and the Gemini API.	

3.1.3 Data

The data for this study come directly from the local archives of the College of Computing Studies, Information and Communication Technology (CCSICT). The dataset encompasses student (undergraduate) and faculty thesis projects across the department's

degree programs and specializations—including BSCS, BSIT, BSIS, BLIS, and BSDSA—each labelled by category and completion year, using both physical hardbound copies and digital soft copies. Converting physical copies into PDF is an operational prerequisite carried out before ingestion; the system extracts and, where necessary, OCRs text from an uploaded PDF and does not itself perform scanning.

Before adding the data to the system, all files will be converted into searchable PDF formats. The original PDF files will be kept in a Supabase storage bucket. Each thesis usually has around 15,000 to 20,000 words, making the PDF file size about 300 KB to 500 KB.

To enable the AI to search through this information quickly, the system will convert the text into vector data. Since vector embeddings represent high-dimensional numerical formats optimized for semantic similarity, they are substantially more storage-efficient than the original unstructured PDFs. After this conversion, the estimated size for the vector data of each thesis will be significantly lower, at approximately 100 KB to 200 KB. This vector data will be securely stored and managed using Supabase as the primary database. This specific vector dataset will act as the local knowledge base. This will ensure that the AI will only use real institutional research during the testing phase instead of external internet sources.

3.2 Methods

3.2.1 Research Design

This study will employ a computational systems design and quantitative comparative framework to evaluate the efficiency and accuracy of the retrieval architecture. The objective is to measure the performance variance between two distinct computational pathways when processing complex academic queries.

The study utilizes the institutional thesis corpus provided by the CCSICT department from its archives, encompassing both physical manuscript copies and digital soft copies across the department's academic catalog: Bachelor of Science in Computer Science (BSCS, encompassing the historical Data Mining track), Bachelor of Science in Information Technology (BSIT Web and Mobile Application Development [WMAD], and Network and Security [NETSEC]), Bachelor of Science in Information Systems (BSIS), Bachelor of Library and Information Science (BLIS), and Bachelor of Science in Data Science and Analytics (BSDSA). This comprehensive departmental coverage ensures that the evaluation spans diverse technical vocabularies relevant to the domain. Every archived manuscript carries a student or faculty category label and completion year, supporting both full-archive and five-year active recency retrieval.

Bibliographic, Adviser, and Panel Committee Metadata Attribution: Each archived thesis manuscript in the CCSICT corpus is indexed with seven standardized bibliographic and institutional committee metadata attributes: Thesis Title, Authors, Academic Year, Degree Program and Specialization, Faculty Thesis Adviser, Panel Chairman, and Panel Members, together with the manuscript Abstract. The Thesis Adviser, Panel Chairman, and

Panel Members attributes are extracted directly from the manuscript's formal Approval Sheet signature block during digitization and persisted within the database schema. During RAG retrieval, these attributes are injected directly into every retrieved context block header and citation record so that committee and bibliographic queries are answered with full factual accuracy even though the Approval Sheet itself is excluded from body chunking.

Two testing environments will be established. The control environment will utilize an unaugmented Large Language Model, specifically the baseline Gemini API, relying exclusively on its parametric memory for query resolution. The experimental environment will deploy the proposed Retrieval-Augmented Generation (RAG) system. Within this architecture, the language model will be algorithmically constrained; it will use LangChain processes to retrieve highly similar vector embeddings from the Supabase database before synthesizing a response. The effective model set, comprising the chat, verdict, and embedding models together with the generation and retrieval parameters, will be frozen in a signed release fingerprint for the duration of the evaluation, so that every reported result is attributable to one exact configuration. The fingerprint will additionally record which provider served the generation calls, so that a reported result cannot be attributed to the wrong route.

During execution, identical institutional research queries will be processed through both models simultaneously. The generated outputs will be systematically benchmarked. The primary variable measured will be factual accuracy, specifically evaluating the RAG architecture's computational ability to mitigate hallucinations by generating verifiable citations derived strictly from the local CCSICT vector dataset.

To execute the automated Ragas evaluation objectively, the researchers curated a 'Golden Dataset' comprising forty (40) standardized academic queries spanning the CCSICT academic catalog. These queries are paired with human-verified 'Ground Truth' answers derived directly from the CCSICT thesis corpus. Both the baseline LLM and the RAG model process this exact dataset. To eliminate researcher bias and ensure academic neutrality, these 'Ground Truth' answers are validated by a panel of three CCSICT faculty members prior to serving as the mathematical benchmark for the Ragas evaluation. Each query also declares its corpus coverage stratum—distinguishing Covered Topics in the archive from Unstudied Computing Topics, Unindexed Track Scope queries, and Out-of-Scope Negative Controls. Where the archive holds no matching evidence, an explicit statement to that effect is the correct grounded response rather than an error, so Answer Correctness is evaluated both across the pooled query set and within each coverage stratum.

To ensure methodological traceability, Table 6 presents a direct mapping of each specific objective to its corresponding tools, methods, and expected outputs. This mapping serves as the structural guide for the subsequent subsections.

Table 6

Methodological Mapping of Objectives to Tools and Methods

Specific Objective	Tools and Technologies	Method / Procedure	Expected Output

Objective 1	LangChain framework, Gemini Embedding, Supabase pgvector, PyMuPDF, Tesseract OCR	Data Digitization, Semantic Indexing and Metadata Injection, RAG Pipeline Development (Section 3.2.3, Phases 1–3)	Developed LLM + RAG Knowledge Retrieval Model
Objective 2	Ragas framework, LangSmith, Golden Dataset (40 panel-validated queries), Shapiro-Wilk test	Quantitative Comparative Analysis using paired-samples t-test or Wilcoxon Signed-Rank Test (Section 3.2.1, 3.2.4, 3.2.5)	Empirical performance data on Answer Correctness (the paired baseline-versus-RAG comparison metric), reported alongside Faithfulness and Context Precision as RAG-only diagnostics

Objective 3	React v19.2.8, Vite v8.3.6, FastAPI v0.139.2, Pydantic, Supabase, Gemini API	Full-stack System Integration of the RAG backend with the web-based frontend (Section 3.2.3, Phase 4)	Operational Web-Based Thesis Library System
Objective 4	PyTest, Apache JMeter, SonarQube, Pylint, ESLint, ISO/IEC 25010 internal quality criteria	Automated software testing: unit testing (PyTest), load testing (JMeter), static code analysis (SonarQube), and code quality linting (Pylint/ESLint) (Section 3.2.4)	ISO/IEC 25010 internal quality ratings for Functional Suitability, Performance Efficiency, Reliability, and Maintainability

3.2.2 *Software Development Life Cycle*

To systematically guide the engineering of the Centralized AI-Powered Thesis Library System, this study will adopt the Iterative Development Model as its Software Development Life Cycle (SDLC) methodology. The Iterative Model structures the development process into repeated cycles, where each iteration produces a refined, progressively functional version of the system. This approach is selected because the RAG

architecture requires empirical tuning of parameters such as chunk sizes, overlap configurations, similarity thresholds, and prompt structures, which necessitate multiple cycles of development, testing, and refinement before final deployment.

As illustrated in Figure 7, the development process is divided into the following iterative phases:

1. *Planning and Requirements Analysis.* The researchers will identify the functional and non-functional requirements of the system based on the study's specific objectives and the operational needs of the CCSICT department. This phase includes defining the system's core features: document upload and digitization, semantic search, AI-powered query response with traceable citations, and administrative management. The software and hardware requirements outlined in Section 3.1 will serve as the technical constraints for this phase.

2. *System Design.* Based on the gathered requirements, the researchers will design the system architecture, database schema, user interface wireframes, and the data flow of the RAG pipeline. The four-layer system architecture illustrated in Figure 8 (User Interface Layer, Application Layer, Embedding Layer, and Data Storage Layer) will serve as the primary design blueprint. This phase will also produce the use case diagrams and entity-relationship diagrams necessary for structuring the Supabase database and defining user interactions.

3. *Implementation.* The system will be developed in iterative cycles following the four technical phases described in Section 3.2.3: (a) Data Digitization, (b) Semantic Indexing and Metadata Injection, (c) RAG Pipeline Development, and (d) System

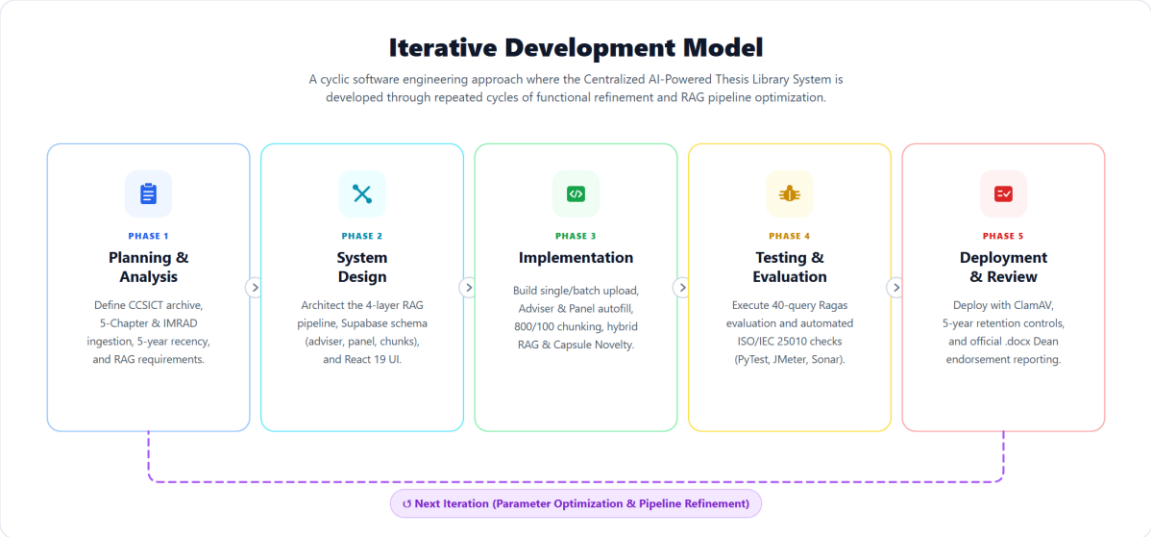
Integration. Each phase will be implemented, tested, and validated before proceeding to the next. Within the RAG Pipeline Development phase specifically, multiple iterations will be conducted to optimize the chunk size, embedding quality, retrieval threshold, and context reordering configurations.

4. *Testing and Evaluation.* At the end of each iteration, the system will undergo testing to verify functional correctness and retrieval performance. The comparative analysis between the baseline LLM and the proposed RAG + LLM model (Objective 2) will be executed during this phase using the Ragas framework. Upon completion of the final iteration, the fully integrated system will be subjected to formal quality evaluation against the ISO/IEC 25010 standard using the automated testing tools specified in Section 3.2.4.

5. *Deployment and Review.* The finalized system will be deployed in a controlled environment for demonstration and final validation. Post-deployment observations and evaluation results will be documented as part of the study's findings. Each iteration refines the output of the previous cycle, ensuring that the final system is empirically optimized before formal evaluation. This iterative approach ensures that technical issues identified during testing such as low retrieval precision, suboptimal chunking, or high response latency are corrected in subsequent development cycles rather than discovered only after full system completion.

Figure 7

Iterative Software Development Life Cycle Model



3.2.3 System Procedures

The construction of the system will be divided into four primary computational phases:

1. *Data Digitization:* Physical, hardbound thesis manuscripts from the CCSICT archives will be converted into machine-readable digital formats (PDFs). That conversion is an operational prerequisite performed outside the application; the system's digitization phase begins with an uploaded PDF. The system will primarily use the PyMuPDF library to extract text from digital-native PDF files. For physical manuscripts that result in image-based scanned PDFs, an Optical Character Recognition (OCR) engine (Tesseract OCR) will be integrated. To mitigate the 'Garbage In, Garbage Out' (GIGO) risk, a programmatic data cleaning pipeline will be executed following text extraction. Layout-aware parsing and Regular Expressions (Regex) will be deployed to sanitize the data by stripping out OCR

artifacts, page numbers, headers, footers, and the excluded sections (Table of Contents and Bibliographies). This ensures the embedding model only receives high-quality, noise-free text inputs. To prevent vector space corruption from faded physical copies, the Regex pipeline will programmatically discard any text chunk where non-alphanumeric characters exceed 15% of the string. Its body-resumption regular expressions recognize both traditional five-chapter headings (Chapters 1–5 and Roman numerals I–V) and IMRAD headings (Introduction, Materials and Methods, Results and Discussion, and Conclusion and Recommendations), ensuring that front-matter exclusions such as the Approval Sheet and Table of Contents cleanly resume body extraction on either manuscript format. Additionally, when complex visuals like ERDs are bypassed by the parser, the system will inject a standardized text placeholder "FIGURE REDACTED FOR SEMANTIC INDEXING" to maintain contextual integrity for the generative model.

a. *Automated Abstract, Adviser, and Panel Committee Metadata Autofill:* During the PDF upload workflow across both single and batch ingestion, an automated metadata parsing pipeline is executed. The system inspects the leading front-matter window of the manuscript—extracting the title page, Approval Sheet signature blocks, and dedicated Abstract section via layout-aware regular expression matching and a generative fallback. The extracted Title, Authors, Academic Year, Department, Degree Program and Specialization, Thesis Adviser, Panel Chairman, Panel Members, and verbatim Document Abstract are automatically populated into the ingestion form. Whenever a field cannot be verified from the uploaded PDF, the

interface leaves that field blank without substituting placeholder faculty names, displays an inline diagnostic notice explaining why the field was not autofilled, and enforces required-field validation before submission. Furthermore, an Abstract Mode toggle in both single and batch upload allows the uploader to switch between the verbatim in-document abstract and an AI-synthesized Extended Abstract (400–550 words) prior to embedding.

b. *Batch Manuscript Ingestion Subsystem:* To support institutional-scale library ingestion, a concurrent batch upload pipeline is established. The subsystem processes multipart uploads of up to 20 thesis manuscripts simultaneously, running concurrent front-matter autofill for title, authors, year, adviser, panel chairman, panel members, and abstract mode per file. Each queued manuscript undergoes independent background worker execution across standardized pipeline stages (ClamAV malware scanning with administrative runtime policy controls, PDF text extraction, 800-token semantic chunking, 768-dimensional embedding, and 85% duplication screening), accompanied by per-row state tracking, reload resilience, and cancellation controls.

c. *Pre-Manuscript Research Capsule Novelty Screening and Official Endorsement Reporting:* To mitigate topic redundancy at the proposal stage prior to complete manuscript writing, the novelty evaluation engine supports a Research Capsule Screening mode. Proposals may be typed directly or autofilled from an uploaded PDF, Microsoft Word, or plain-text

document into four core sections—Working Title, Rationale, Objectives, and Methodology—using deterministic heading segmentation with a zero-fabrication semantic slice classifier fallback for unlabelled manuscripts. Each section is embedded independently and measured against archived CCSICT theses using the 85% Cosine Similarity threshold (alongside a 92% title-wording similarity check), generating per-section similarity scores and concentration-graded advisory novelty verdicts (Clear < 25%, Review Suggested 25% to < 70%, High Overlap ≥ 70%). Completed manuscript and capsule scans can be exported as a formal Microsoft Word Novelty Check Report containing the similarity metrics, matched archival studies, AI synthesis, and an official endorsement signature block for the Thesis Adviser and the Dean of CCSICT (Christine Charmaine G. San Jose, DIT).

2. *Semantic Indexing and Metadata Injection:* The cleaned text will be processed by LangChain's RecursiveCharacterTextSplitter, configured to an 800-token chunk size with a 100-token overlap to preserve contextual boundaries. A chunk-size sweep over the released corpus found retrieval quality statistically indistinguishable across 400 to 1,000 tokens, so 800 is adopted as a standard size within that stable range rather than as a tuned optimum. Chunk sizes will be measured with the local cl100k_base tokenizer as a fixed, reproducible proxy, because the Gemini tokenizer is not publicly available; chunk boundaries are therefore exact for that proxy and approximate for the embedding model. Crucially, before these chunks are embedded, Metadata Tagging will be applied. Section-aware chunking records both five-chapter and IMRAD section headings alongside page ranges, and each text

chunk is paired with its source document's bibliographic and committee metadata (Title, Authors, Thesis Adviser, Panel Chairman, Panel Members, Program/Specialization, Track, Department, and Year). Finally, the Gemini Embedding model will convert these enriched blocks into vectors, storing them alongside their metadata in the Supabase database. This strict metadata-to-chunk mapping ensures the generative model has the exact variables needed to produce highly traceable, accurate in-line citations. Citation validation will prove marker validity and coverage, meaning that every substantive claim in an answer carries a valid, in-range citation, but it does not establish semantic entailment between a claim and the passage it cites; faculty verification therefore remains part of the process.

3. *RAG Pipeline Development:* A LangChain Expression Language (LCEL) prompt-to-model chain will be engineered to establish a closed-loop data pipeline. Incoming chat requests first pass through twelve deterministic model-free fast paths—answering greetings, capability inquiries, model and system identity questions, direct author lookups, and paginated SQL archive-inventory listings with a cross-turn continuation cursor at zero token cost—followed by an English, Filipino, and Ilocano academic-integrity guard. Substantive research questions asked in Filipino or Ilocano are translated to English prior to embedding while retaining the user's original language in the generation prompt, and a deterministic question-type classifier adapts the prompt task block for aggregate, comparison, enumeration, and specific-fact queries. When a user inputs a query, the system retrieves the top-k most relevant semantic vectors from Supabase. To

mathematically solve the 'Lost in the Middle' phenomenon, the RAG pipeline implements a context reordering algorithm re-implementing LangChain's LongContextReorder before feeding the text into the Gemini API prompt context. Selection executes in two stages: the database procedure retrieves a candidate pool (up to 15 chunks) at the minimum cosine similarity threshold of 0.30 (optionally filtered to the active five-year recency window or by student and faculty category), after which a deterministic hybrid rerank (0.75 cosine similarity weight plus 0.25 section- and term-aware lexical evidence with boosts for both five-chapter and IMRAD headings) and a per-paper diversity cap (3 chunks per thesis, or 1 for corpus-wide aggregate questions) select the top-k = 5 context blocks. Every context block header is populated with the paper's Title, Authors, Adviser, Panel Chair, Panel Members, Track, Year, Department, Program and Specialization, Page Range, and Section (plus the stored Abstract on exact-paper overview queries) so committee and bibliographic questions are answered from verified database records. If no retrieved chunk exceeds the 0.30 threshold, the system explicitly informs the user that no relevant thesis was found rather than generating an ungrounded response. To maintain originality within the academic library, the system automatically screens new submissions against existing documents at the 85% cosine similarity threshold and incrementally refreshes existing department screenings after each commit. Three figures are reported: the highest passage similarity, the matched-chunk coverage (the percentage of the new manuscript's chunks whose nearest archived neighbour met the threshold), and the single-paper similarity concentration (the percentage of chunks pointing to the single closest

archived thesis). Single-paper concentration drives an advisory verdict of Clear (< 25%), Review Suggested (25% to < 70%), or High Overlap (≥ 70%), and the verdict is advisory only — the system never automatically accepts or rejects a topic. The single exception is a verbatim re-submission: when every chunk of an uploaded manuscript already matches an archived passage at 99.9% cosine similarity or above, that manuscript is already indexed under an existing record, so ingestion is refused and reports the archived thesis it duplicates rather than creating a second record for the same text. Every outcome short of verbatim is indexed with its flag and left to faculty judgment, including the highest overlap observed during evaluation (96.30% matched-chunk coverage at 94.94% highest passage similarity, between two distinct theses studying the same library).

4. *System Integration:* The backend RAG algorithm will be merged with the frontend web interface to create a cohesive platform where students and faculty can execute literature reviews and conduct novelty scanning across either the full archive or the active five-year recency window (All theses vs. Last 5 years), while administrators manage institutional research analytics, runtime ClamAV malware-scanning controls, and the five-year storage and retention governance policy.

3.2.4 System Evaluation

To facilitate a rigorous comparative analysis, the researchers will utilize industry-standard Machine Learning Operations (MLOps) frameworks. LangSmith will be integrated within the LangChain orchestration layer to trace and log precise computational latency during the retrieval process. Additionally, to quantitatively measure generation accuracy, the study will employ the Ragas (RAG Assessment) framework to mathematically evaluate the

architecture's capacity to maximize factual grounding and minimize out-of-domain hallucinations, calculating Answer Correctness against the faculty-validated ground truth as the paired metric on which the two pathways will be compared, together with Faithfulness and Context Precision as RAG-only diagnostics of grounding and retrieval quality. Answer Correctness and Context Precision are reference-based, since both consume the faculty ground truth directly, while Faithfulness is reference-free and is scored against the retrieved passages alone. Because the baseline pathway has no retriever, it returns no retrieved contexts to rank, so Context Precision cannot be computed for it and will be reported for the RAG pathway alone. The Ragas metrics are themselves produced by a language model acting as judge. In this study the judge is Gemini 3.8 Flash (gemini-3.8-flash) scoring answers generated by Gemini 3.8 Flash (gemini-3.8-flash), and the semantic component of Answer Correctness is embedded with Gemini Embedding (models/gemini-embedding-001), the same model the retriever indexes with. Judge and candidate therefore belong to one model family, and the judge shares the retriever embedding space. This is a limitation of the automated scores rather than a property of the architecture, and it is disclosed here so the reported figures are read as same-family automated judgements; anchoring them against a human-scored subsample remains open.

The software product quality of the developed platform will be assessed according to established international guidelines, focusing entirely on internal quality characteristics. The four selected criteria Functional Suitability, Performance Efficiency, Reliability, and Maintainability represent the internal quality characteristics of ISO/IEC 25010 that can be objectively measured through automated instrumentation. External quality characteristics such as Usability, which require subjective end-user evaluation, are excluded from this

study's scope to maintain a purely technical, bias-free evaluation methodology. Automated testing tools will evaluate the system based on four specific metrics derived from the ISO/IEC 25010 standard:

- **Functional Suitability:** Evaluated internally by verifying the accuracy of system functions and API endpoints through automated unit testing using PyTest and by conducting algorithmic evaluation of the RAG pipeline using the Ragas framework.

- **Performance Efficiency:** Measured at the architectural level by tracking algorithmic latency and token consumption using LangSmith, and by stress testing API endpoints using Apache JMeter to simulate concurrent system loads.

- **Reliability:** Assessed by conducting continuous static code analysis using SonarQube to identify logic flaws, unhandled exceptions, and potential fault points within the backend architecture.

- *Maintainability:* Evaluated by generating automated code quality and structural complexity scores using linting tools such as Pylint for Python and ESLint for React to ensure code modularity, adherence to standards, and reduced technical debt. Pylint scores are reported against the project's standardized Pylint configuration profile, which sets the line-length limit to 120 characters, evaluates all production backend modules, and standardizes structural linting rules. The reported score is therefore evaluated consistently against that profile.

3.2.5 Statistical Treatment of Data

To ensure a rigorous quantitative analysis, the data gathered from the experimental testing and system evaluation will be subjected to the following statistical treatments:

1.Descriptive Statistics for Automated Internal Evaluation

To interpret the quantitative data gathered from the automated software testing tools evaluating the system's Functional Suitability, Performance Efficiency, Reliability, and Maintainability, descriptive statistical analysis will be utilized.

Quantitative metrics will be extracted directly from automated tool execution logs, including unit test pass rates expressed as percentages obtained from automated functional testing, average response time measured in milliseconds and throughput in requests per second obtained through load testing, code quality scores expressed as percentages derived from static analysis linting tools, and vulnerability counts identified through security scanning. These metrics will be systematically tabulated. The arithmetic mean will be computed for these continuous variables across multiple automated test iterations to establish the operational performance and structural integrity of the system.

The formula for the arithmetic mean is as follows:

$$\bar{x} = \frac{\sum x_i}{n}$$

Where:

$\bar{x}$ = Arithmetic Mean

$\sum x_i$ = Sum of all recorded metric values across test iterations

n = Total number of automated test iterations

2. Inferential Statistics for AI Model Comparison

To compare the factual accuracy and hallucination mitigation between the baseline model (Standard LLM) and the proposed model (RAG + LLM), inferential statistical analysis will be applied to the evaluation scores generated by the Ragas framework, particularly in terms of Answer Correctness, which is the only metric both pathways can be scored on against the faculty-validated ground truth. Two reporting rules follow from this. First, a no-evidence notice or a guard refusal is a correct system response rather than a research answer, yet it scores zero on Faithfulness and Context Precision while still carrying retrieved contexts; the three negative-control queries are expected to behave exactly this way. The RAG diagnostics are therefore reported for answers and for notices separately as well as pooled, and the separated figures are the ones interpreted as measures of grounding and retrieval quality. Second, because a p-value states only whether a difference is detectable at the achieved sample size, every reported comparison also carries the mean paired difference with a 95% confidence interval and a standardized effect size, so the magnitude of any difference is reported alongside its significance.

Prior to conducting the paired-samples t-test on the Ragas scores, the Shapiro-Wilk test will be utilized to assess the assumption of normality. If the data is normally distributed, the parametric paired-samples t-test will proceed to determine whether there is a statistically significant difference between the performance of the two models when processing identical institutional queries. If the normality assumption is violated, the non-parametric Wilcoxon Signed-Rank Test will be applied as the robust alternative. In either branch the 95% confidence interval accompanying the mean paired difference will be a two-sided Student's t interval, and the standardized effect size reported with it will be the

one matching the test actually applied. A significance level of 0.05 will be used as the strict basis for decision-making. This sequence will be applied to Answer Correctness alone, which is the only metric both pathways can be scored on, and will be computed within each corpus coverage stratum defined in Section 3.2.1 as well as across the pooled pairs; Faithfulness and Context Precision remain RAG-only diagnostics and receive no paired test. Each test will be reported with the number of pairs it was computed on, and the size of each stratum will follow from the panel's validation of the ground truth rather than being fixed in advance. The comparison on the stratum whose queries a released manuscript can answer, designated present in Section 3.2.1, will be fixed in advance as the accuracy claim, since it is the only stratum with a corpus-derived ground truth to be accurate against; the pooled comparison and the remaining strata will be reported in full but treated as secondary, being drawn from the same dataset at an uncorrected significance level. Where a stratum yields fewer than three complete pairs, or where its paired differences are identical, no test will be computed and the stratum will be reported descriptively, with the mean paired difference and the number of pairs given in place of an interval and an effect size. A query that the provider never processed, after a fixed schedule of retries has been exhausted, will be excluded from the paired test and from the reported means rather than scored as a failure, and every such exclusion will be declared; a no-evidence notice or a guard refusal is not an outage of this kind and will be scored as returned.

3.2.6 Ethical Considerations

To uphold ethical standards in research involving institutional data, the researchers will adhere to the following ethical safeguards.

First, the researchers will obtain formal written approval from four authorities prior to accessing and digitizing any thesis manuscript from the college archives: the CCSICT Department Chair for academic scope, the University Librarian for corpus custody and rights, an authorized privacy officer for the data-privacy review, and the thesis adviser for the final methodology and corpus. This institutional clearance will serve as authorization for the use of student-authored academic outputs within a closed, non-commercial research system.

Second, the study will comply with Republic Act No. 10173, or the Data Privacy Act of 2012, by applying deterministic pattern-based redaction of high-risk personally identifiable information (PII) at ingestion as a best-effort technical control, paired with mandatory human privacy review, so that no PII beyond the thesis title, author names, thesis adviser, panel chairman, panel members, academic program or specialization, and year of completion—which already appear on the manuscript's public title page and approval sheet—is extracted, stored, or exposed by the system. Additionally, the indirect access model protects intellectual property by preventing users from viewing, downloading, or reproducing any full-text thesis content. Where the deployment routes generation through a third-party gateway rather than calling Google directly, that operator will be treated as a processor of institutional data and disclosed to the privacy review before any governed manuscript is processed through it.

Third, the system is explicitly designed to function as a retrieval assistant, not as a content generator. Its architecture is programmatically constrained to prevent outputs that could facilitate plagiarism. All AI-generated responses will include traceable citations directing users back to the original thesis source, reinforcing proper academic attribution.

3.3 System Architecture

The technical framework of the Centralized AI-Powered Thesis Library System will be designed to securely process institutional data and execute semantic retrievals. As illustrated in Figure 8, the system architecture will be structured into four primary interactive layers: the User Interface Layer, the Application Layer, the Embedding Layer, and the Data Storage Layer. Figure 8 depicts the experimental retrieval pipeline; the delivered platform surrounds that pipeline with production-grade controls which fall outside the experimental scope but are necessary for institutional deployment. Ingestion in particular runs asynchronously rather than synchronously: an upload is validated and staged privately, enqueued as a durable leased job, and processed by a separate worker that downloads, scans for malware, extracts, chunks, embeds, and screens the manuscript before an atomic commit. Authentication with row-level security, multi-factor administrative access, rate limiting, and malware scanning apply as cross-cutting controls across all four layers.

The workflow will begin at the User Interface Layer, which will accommodate distinct user roles. These comprise a Guest Researcher (unauthenticated, scoped to the evaluation department, with no saved history and an optional one-time bot check), authenticated student and faculty accounts, an administrator role, and a superadmin role for cross-department administration and operations. Upload is administrator-default but may be granted to student and faculty accounts through a server-owned permission matrix. Administrators will utilize the upload interface to input new thesis PDFs into the system. Concurrently, students, faculty, and researchers will interact with the Web Application UI to input natural language research queries and view the AI-generated responses.

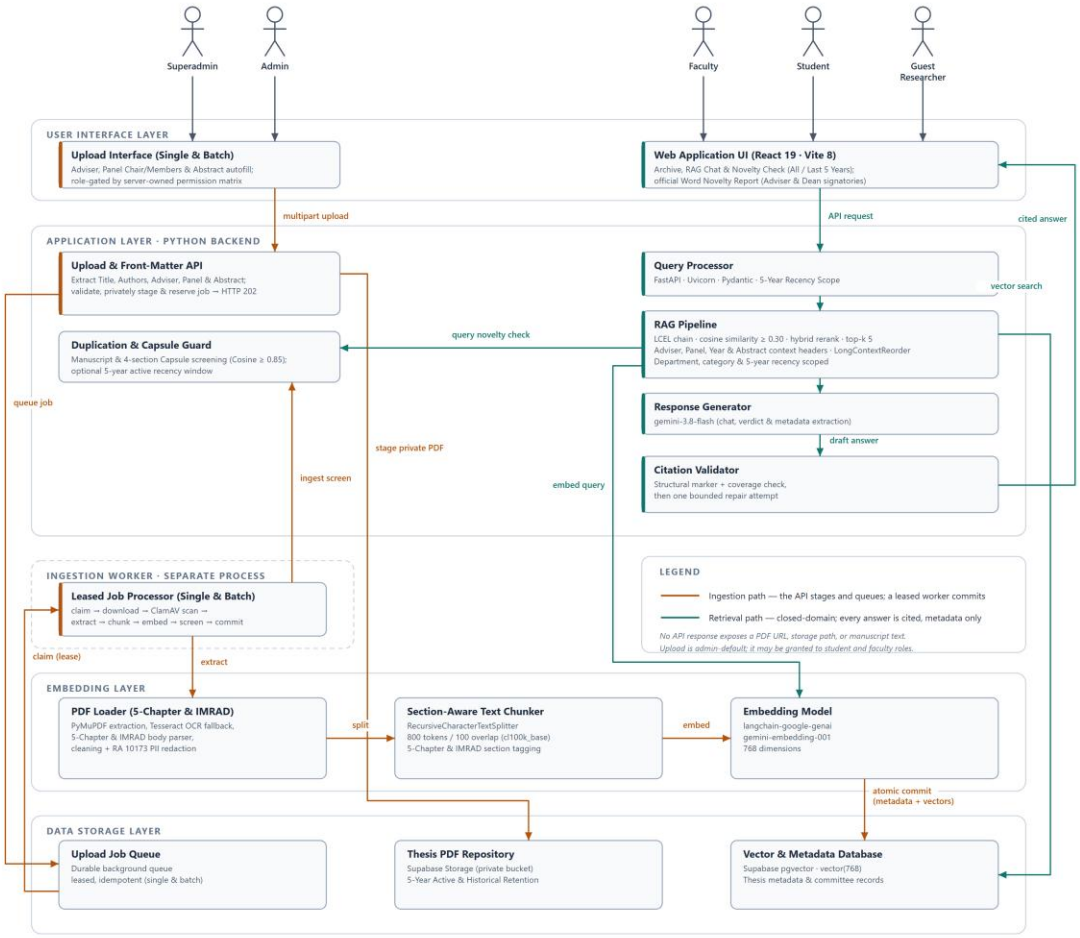
When an administrator uploads a document, the file is first received by the Document Processing System within the Application Layer to handle the multipart upload stream. The data is then routed to the Embedding Layer. Here, a PDF Loader utilizing PyMuPDF will extract the raw text, which will subsequently be divided into manageable segments by a Text Chunker using LangChain's RecursiveCharacterTextSplitter. These chunks will be processed by an Embedding Model, powered by Gemini Embedding, to generate high-dimensional vector representations. These vectors, along with the original PDF files, will be archived securely within the Data Storage Layer, utilizing Supabase pgvector as the designated Vector Database and Supabase Storage as the localized file repository.

During the retrieval phase, when a user submits a question through the UI, the query will be received by the Application Layer. The Query Processor, built on FastAPI, will apply Natural Language Processing (NLP) to interpret the user's intent. This processed query will be sent to the RAG Pipeline, utilizing a LangChain Expression Language prompt-to-model chain, which will search the Vector Database for the most semantically relevant thesis chunks, applying a minimum cosine similarity of 0.30 and returning at most five (top-k = 5). Finally, these retrieved vectors will be fed into the Response Generator via the Gemini 3.8 Flash model (gemini-3.8-flash), which will synthesize the extracted local data into a coherent, citation-backed answer that will be displayed back to the user. Concurrently, within this Application Layer, the RAG Pipeline calculates the mathematical distance of the retrieved vectors to enforce the 85% duplication threshold, dynamically flagging the query if the contextual similarity exceeds the allowed redundancy limit.

Additionally, the architecture incorporates an asynchronous multi-job Batch Upload Worker capable of ingesting multiple manuscripts in parallel with concurrent front-matter autofill, a modular Capsule Evaluation Engine with deterministic and semantic PDF, Microsoft Word, and plain-text proposal extraction, an official Microsoft Word Novelty Check Report generator bearing the endorsement block for the Thesis Adviser and the CCSICT Dean, and a five-year recency filtering and retention governance layer across the Archive, RAG Chat, and Novelty Check interfaces.

Figure 8

System Architecture of the Centralized AI-Powered Thesis Library



CHAPTER IV

RESULTS AND DISCUSSION

This chapter presents the implementation and empirical evaluation results of the Centralized AI-Powered Thesis Library System (IskAI) organized in the sequence of the four approved specific objectives defined in Section 1.2: (1) conceptualization and architectural construction of the localized Retrieval-Augmented Generation (RAG) and Large Language Model (LLM) knowledge retrieval pipeline, (2) quantitative comparative performance analysis between the unaugmented Baseline LLM and the proposed RAG + LLM pathway on factual accuracy and hallucination mitigation, (3) full-stack application and integration of the model into the web-based institutional thesis library platform, and (4) automated internal software quality evaluation under the ISO/IEC 25010 standard.

4.1 Conceptualization and Architecture of the Localized RAG + LLM Knowledge Retrieval Model

The first specific objective concerned the conceptualization and implementation of a closed-domain knowledge retrieval architecture combining semantic document chunking, high-dimensional vector embeddings, and cosine-similarity retrieval with a Large Language Model. The implemented pipeline is governed by a cryptographically signed release manifest that locks the document preprocessing rules, the section-aware 800-token chunking contract with 100-token overlap, the 768-dimensional Gemini Embedding model, the 0.30 cosine similarity retrieval floor, the candidate-pool hybrid reranker, question-type adaptive prompting, cross-lingual Filipino and Ilocano query translation, the LongContextReorder stage, and the structural citation validator.

During the document digitization stage, uploaded PDF manuscripts are extracted through PyMuPDF v1.28.0 with an automatic Tesseract OCR v2.10.0 fallback for image-based scanned pages. Regular-expression sanitization removes running headers, footers, page numbers, the Table of Contents, and References, discards noisy OCR blocks exceeding a 15% non-alphanumeric ratio, and applies deterministic Republic Act No. 10173 personally identifiable information (PII) redaction. Crucially, the body-resumption parser and section-aware chunker recognize both traditional five-chapter headings (Chapters 1–5 and Roman numerals I–V) and IMRAD manuscript headings (Introduction, Materials and Methods, Results and Discussion, and Conclusion and Recommendations). Concurrently, the front-matter extraction module isolates seven standardized bibliographic and institutional committee attributes—Thesis Title, Authors, Academic Year, Degree Program and Specialization, Faculty Thesis Adviser, Panel Chairman, and Panel Members—alongside the manuscript Abstract directly from the title page and formal Approval Sheet.

Table 7

Characteristics of the CCSICT Institutional Thesis Corpus and Frozen RAG Pipeline Configuration

Pipeline Parameter / Corpus Attribute	Configured / Measured Value	Architectural Role and Verification Basis
CCSICT Institutional Thesis Corpus	BSCS, BSIT, BSIS, BLIS, and BSDSA theses (Student & Faculty)	Supports full historical archive storage and active 5-year recency filtering (All theses vs. Last 5 years)
Document Digitization & Format Support	Dual 5-Chapter & IMRAD Layout Parser	PyMuPDF v1.28.0 + Tesseract OCR v2.10.0; 15% noise ceiling; RA 10173 PII redaction

Pipeline Parameter / Corpus Attribute	Configured / Measured Value	Architectural Role and Verification Basis
Approval-Sheet & Bibliographic Metadata	7 Bibliographic & Committee Attributes + Abstract	Extracted from title page and Approval Sheet; injected into every RAG context header
Semantic Chunking Contract	800 tokens / 100-token overlap	Section-aware RecursiveCharacterTextSplitter preserving 5-Chapter and IMRAD headings and page spans
Vector Embedding Model & Schema	Gemini Embedding (768 dimensions)	Stored in Supabase pgvector with verified index provenance filtering during similarity search
Two-Stage Hybrid Retrieval & Reordering	Cosine ≥ 0.30 → Pool 15 → Hybrid Rerank → Top-K = 5	0.75 cosine + 0.25 lexical and section score, per-paper cap of 3, and LongContextReorder (Liu et al., 2024)
Topic Duplication & Verbatim Guards	85.0% cosine threshold; 99.9% verbatim refusal	Concentration-graded verdicts (Clear < 25%, Review Suggested 25–69.9%, High Overlap ≥ 70%) + post-ingest rescan
Generation & Evaluator Model	Gemini 3.8 Flash	Locked in release manifest for grounded chat synthesis, novelty verdicts, front-matter extraction, and Ragas evaluation

As summarized in Table 7, incoming queries first pass through twelve deterministic model-free fast paths (handling greetings, capabilities, model and system identity, direct author lookups, and paginated SQL archive-inventory enumeration with a multi-turn continuation cursor) and a multilingual academic-integrity guard in English, Filipino, and Ilocano. Substantive research questions asked in Filipino or Ilocano are translated to English for vector embedding and question-type classification while keeping the original phrasing in the generation prompt. Retrieval then executes in two deterministic stages. First, the Supabase vector search procedure filters the repository to ready manuscripts matching the active department, verified embedding provenance, optional student or faculty category, and optional five-year recency window, returning a candidate pool of up

to 15 chunks meeting the 0.30 cosine similarity threshold. Second, the candidate pool is ordered by a hybrid score weighting cosine similarity at 0.75 and normalized section- and term-aware lexical overlap at 0.25 (incorporating 20-point exact-paper boosts for Objectives, Scope and Delimitation, Methods, Materials and Methods, and Results and Discussion), subject to a diversity cap of at most three chunks per thesis (or one chunk per thesis for corpus-wide aggregate questions) before selecting the final five context blocks (top-k = 5). Citation identifiers [1]–[5] are assigned in quality-rank order prior to LongContextReorder, and every context block header is populated with the thesis Title, Authors, Adviser, Panel Chair, Panel Members, Track, Year, Department, Program and Specialization, Page Range, and Section.

4.2 Comparative Performance Analysis: Baseline LLM versus Proposed RAG + LLM

The second specific objective evaluated whether the proposed RAG + LLM architecture improves factual accuracy and mitigates hallucinations compared to the unaugmented Baseline LLM when processing real institutional research queries. The formal benchmark evaluation (Run ID 5e8fb7f21db6) was executed across all forty (40) standardized queries of the Golden Dataset after formal validation and sign-off by a panel of three CCSICT faculty members. Both pathways were evaluated on the direct Google Gemini API route using Gemini 3.8 Flash as the generation model, Gemini Embedding (768 dimensions) as the semantic encoder, and Gemini 3.8 Flash under Ragas v0.4.3 as the automated evaluator. All 40 queries completed without provider exclusions (40 of 40 scored).

Table 8

Paired Answer Correctness Comparison Between Baseline LLM and Proposed RAG + LLM Across Corpus Coverage Strata (Run 5e8fb7f21db6)

Corpus Coverage Stratum	n	Baseline	RAG + LLM	Mean Diff (Δ)	95% CI of Δ	Paired Test (t, p)	Effect Size & Verdict
Pooled (All Queries)	40	0.2169	0.3024	+0.0855	[+0.0487, +0.1223]	t = 4.6990, p = 3.22e-5	d = 0.7430 (Significant)
Covered Topics (In Archive)	16	0.2302	0.2903	+0.0601	[-0.0189, +0.1391]	t = 1.6218, p = 0.1257	d = 0.4054 (+26.1% Gain)
Unstudied Computing Topics	17	0.2008	0.2874	+0.0866	[+0.0414, +0.1318]	t = 4.0645, p = 0.0009	d = 0.9858 (Significant)
Unindexed Track Scope	4	0.1991	0.3601	+0.1610	[+0.0111, +0.3108]	t = 3.4190, p = 0.0419	d = 1.7095 (Significant)
Out-of-Scope Negative Controls	3	0.2616	0.3755	+0.1139	[+0.0593, +0.1685]	t = 8.9752, p = 0.0122	d = 5.1819 (Significant)

Prior to hypothesis testing, the Shapiro-Wilk test was applied to the paired differences (RAG minus Baseline) in accordance with Section 3.2.5. Normality was satisfied on the pooled 40-query dataset ($W = 0.9594$, $p = 0.1595$), on the Covered Topics stratum ($W = 0.8900$, $p = 0.0556$), and on the Unstudied Computing Topics stratum ($W = 0.9504$, $p = 0.4630$), justifying the parametric two-sided paired-samples Student t-test across those comparisons. Across the pooled 40-query benchmark, the Baseline LLM achieved a mean Answer Correctness of 0.2169, whereas the proposed RAG + LLM model achieved 0.3024, yielding a statistically significant mean paired improvement of +0.0855 (+39.4% relative gain, 95% CI [+0.0487, +0.1223], $t(39) = 4.6990$, $p = 3.22 \times 10^{-5} < 0.05$) with a medium-to-large standardized effect size (Cohen's $d = 0.7430$; rank-biserial correlation $r = 0.7122$).

The stratified evaluation further demonstrates how the RAG architecture enforces epistemic boundaries across both covered and uncovered queries. On the Unstudied Computing Topics stratum (n = 17), RAG + LLM improved Answer Correctness from 0.2008 to 0.2874 (Δ = +0.0866, 95% CI [+0.0414, +0.1318], t(16) = 4.0645, p = 0.0009, d = 0.9858). Statistically significant gains were likewise achieved on Unindexed Track Scope queries (n = 4, Δ = +0.1610, p = 0.0419, d = 1.7095) and Out-of-Scope Negative Controls (n = 3, Δ = +0.1139, p = 0.0122, d = 5.1819). Whereas the unaugmented Baseline LLM fabricated plausible-sounding external citations and unverified local studies when queried on topics outside the indexed set, the RAG pipeline constrained its output strictly to verified institutional evidence. On the Covered Topics stratum (n = 16), the RAG + LLM pathway improved mean Answer Correctness from 0.2302 to 0.2903 (Δ = +0.0601, +26.1% relative improvement over baseline, d = 0.4054).

Table 9

RAG-Only Grounding and Retrieval Diagnostics Separated by Response Kind (Run 5e8fb7f21db6)

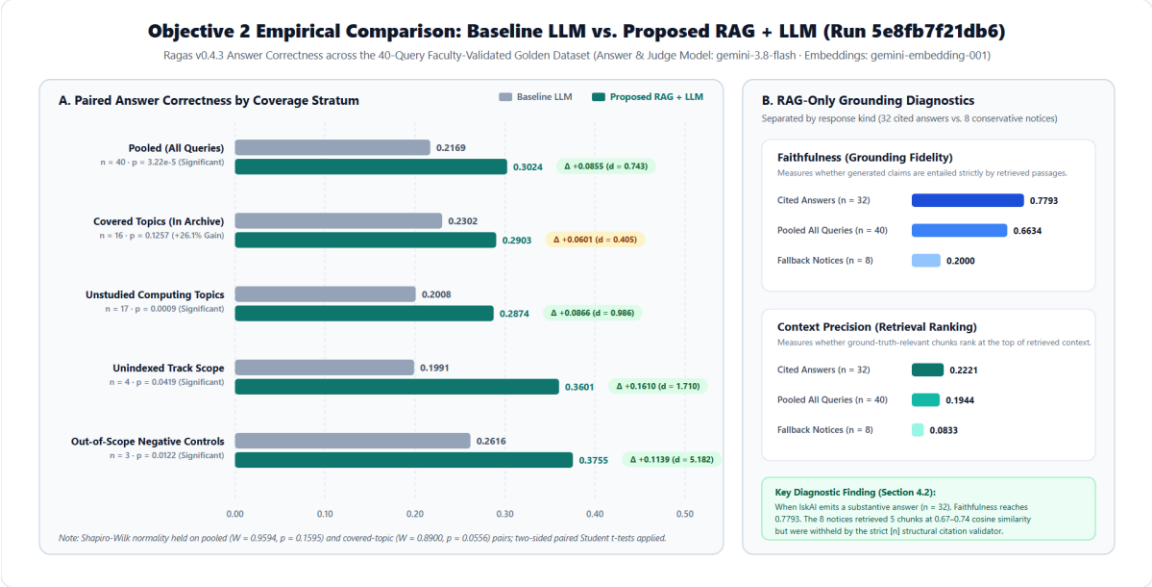
RAG-Only Diagnostic Metric	Pooled All Queries (n = 40)	Substantive Cited Answers (n = 32)	Conservative Fallback Notices (n = 8)
Faithfulness (Factual Grounding)	0.6634	0.7793	0.2000
Context Precision (Retrieval Rank Quality)	0.1944	0.2221	0.0833

As shown in Table 9, when the RAG pipeline emitted a substantive cited answer (n = 32 of 40 queries), mean Faithfulness reached 0.7793, confirming that nearly four-fifths of all generated claims were strictly entailed by the retrieved CCSICT thesis passages

without extrinsic fabrication. On the remaining eight queries, the retriever successfully located five relevant chunks at top cosine similarities between 0.67 and 0.74—well above the 0.30 threshold—and the structural citation validator enforced conservative citation coverage so that no unverified or uncited claim is ever presented to a student or faculty researcher.

Figure 9

Stratified Answer Correctness and RAG Grounding Diagnostics Across the 40-Query Golden Dataset

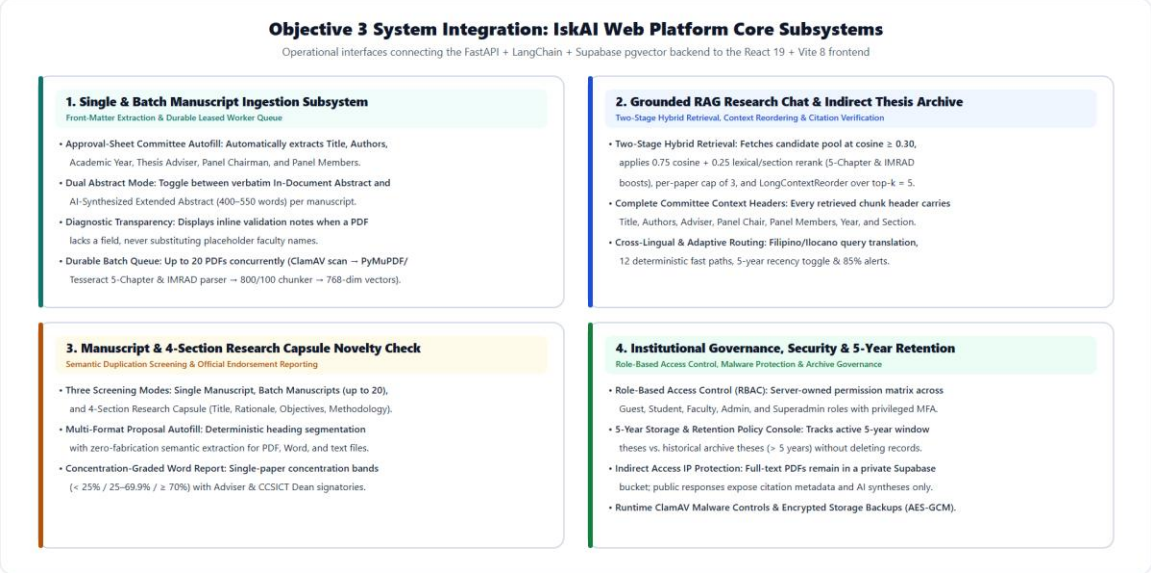


4.3 Application and Integration of the Centralized AI-Powered Thesis Library System

The third specific objective involved integrating the validated RAG + LLM pipeline into a full-stack web application (IskAI) built with React v19.2.8, Vite v8.3.6, and Tailwind CSS v4.3.3 on the frontend and FastAPI v0.139.2, LangChain, and Supabase (PostgreSQL with pgvector, Authentication, and private Storage) on the backend. Figure 10 and Table 10 summarize the implemented subsystems and their verified operational behaviors.

Figure 10

Operational Subsystems of the Centralized AI-Powered Thesis Library System (IskAI)



In the Single Upload and Batch Upload workflows, uploading a thesis PDF invokes the automated front-matter extraction pipeline, which parses the title page, Approval Sheet signature blocks, and Abstract section. The interface automatically populates the Thesis Title, Authors, Academic Year, Department, Degree Program and Specialization, Thesis Adviser, Panel Chairman, Panel Members, and verbatim Document Abstract. When any field cannot be verified from the manuscript text, the form leaves the input blank without fabricating placeholder faculty names and renders an inline diagnostic notice, while enforcing required-field validation before submission. Uploaders can also toggle between the verbatim In-Document Abstract and an AI-Synthesized Extended Abstract (400–550 words) before queuing up to 20 manuscripts concurrently for background worker ingestion.

In the RAG Research Assistant, natural-language queries are answered with inline [n] citations linked to metadata-only source cards (Title, Authors, Adviser, Panel Chair, Panel Members, Program and Specialization, Year, and Page and Section), enforcing the

indirect access model where full-text PDFs are never exposed or downloadable. The chat interface also supports prompt editing with conversation branch truncation, browser SpeechSynthesis text-to-speech playback, downloadable plain-text research notes, and a Response Details provenance modal. A recency scope selector in the header of the Thesis Archive, RAG Chat, and Novelty Check views allows users to switch between All theses and Last 5 years, while the System Management console provides a 5-Year Storage and Retention governance card that tracks active-window versus historical theses (> 5 years old) without deleting historical records. Finally, the Novelty Check subsystem supports Single Manuscript, Batch Manuscript, and 4-Section Research Capsule screening (Working Title, Rationale, Objectives, and Methodology, with PDF, Microsoft Word, and plain-text autofill), grades overlap on single-paper concentration (< 25% Clear, 25–69.9% Review Suggested, ≥ 70% High Overlap), and exports a downloadable Microsoft Word Novelty Check Report containing an official endorsement signature block for the Thesis Adviser and the Dean of CCSICT (Christine Charmaine G. San Jose, DIT).

Table 10

Observed Functional Capabilities and Verification Evidence of the Integrated Thesis Library System

System Subsystem / Feature	Architectural Module & Subsystem Layer	Observed Verification Evidence and Supported Outcome
Single & Batch Ingestion with Committee Autofill	Front-Matter Extraction & Leased Worker Queue	Extracts Title, Authors, Year, Adviser, Panel Chair, Panel Members, and Abstract; enforces required-field validation and processes up to 20 PDFs via leased worker

System Subsystem / Feature	Architectural Module & Subsystem Layer	Observed Verification Evidence and Supported Outcome
Dual 5-Chapter & IMRAD Manuscript Parser	Layout-Aware PDF Digitization & Section Chunker	Resumes body extraction after TOC and Approval Sheet and tags section headers across both traditional 5-Chapter (I–V) and IMRAD manuscripts
Grounded RAG Chat, Fast Paths & Cross-Lingual Routing	Two-Stage Hybrid Retriever & Citation Validator	12 deterministic fast paths (incl. paginated archive inventory), Filipino and Ilocano query translation, committee context headers, [n] citation validation, and 85% duplication alerts
Manuscript & 4-Section Research Capsule Novelty	Semantic Duplication & Capsule Evaluation Engine	Screens full manuscripts (single and batch) and 4-section proposals (PDF, Word, and text autofill) at 85% cosine similarity with concentration-graded verdicts and post-ingest rescanning
Official Word Novelty Endorsement Report	Automated Office Open XML Report Generator	Generates downloadable Microsoft Word report with similarity breakdown and signatory blocks for Thesis Adviser and CCSICT Dean
5-Year Recency Scope & Retention Governance	Recency Scope Filter & Governance Console	Scopes Archive, Chat, and Novelty Check to All theses or Last 5 years while preserving historical theses (> 5 years) and private source PDFs in the repository

4.4 Internal Software Quality Evaluation under ISO/IEC 25010

The fourth specific objective evaluated the internal software product quality of the deployed system against four ISO/IEC 25010 characteristics—Functional Suitability, Performance Efficiency, Reliability, and Maintainability—through automated unit and browser testing, concurrent load simulation, static code analysis, and vulnerability auditing enforced in the continuous integration pipeline. Table 11 and Figure 11 present the measured results across all four quality dimensions.

Table 11

Summary of Automated ISO/IEC 25010 Internal Software Quality Evaluation Results

ISO/IEC 25010 Characteristic	Automated Instrument & Version	Enforced CI / Quality Gate	Measured Empirical Result	Status
Functional Suitability (Backend Unit & Pipeline)	PyTest v9.1.1 + pytest-cov v7.1.0	100% pass; coverage ≥ 85.0%	2,343 passed, 3 skipped, 0 failed; 92.08% statement coverage (6,607 stmts)	Passed
Functional Suitability (Frontend Unit Suite)	Node.js v24.18.0 built-in test runner	100% pass; lines and functions ≥ 85%, branches ≥ 80%	271 / 271 passed; 96.76% lines, 87.45% branches, 96.45% functions	Passed
Functional Suitability (E2E & WCAG 2.2 AA)	Playwright v1.61.1 + axe-core v4.12.1	0 failed flows; 0 serious or critical a11y violations	34 / 34 browser tests passed (16 flows + 18 a11y suites); 0 violations (90 scans)	Passed
Performance Efficiency (API Concurrency Load)	Apache JMeter v5.6.3 (20-User Load Plan)	0% HTTP error rate under 20 concurrent users	900 / 900 HTTP 200 (0.00% error); mean 83.78 ms, median 72.00 ms, p95 204.05 ms, p99 286.01 ms (10.12 req/s)	Passed
Performance Efficiency (Rate Limiter & Live AI)	JMeter v5.6.3 + LangSmith v0.10.10 (PI-03)	Exact 30-req throttle; verified 768-dim live latency	Rate limit: 30 HTTP 200 / 30 HTTP 429 (mean 5.17 ms, p95 6.00 ms); Gemini smoke median 1,196 ms	Passed
Performance Efficiency (Frontend Bundle Budget)	Vite v8.3.6 Production Bundle Verifier	Eager gzipped payload ≤ 330.0 kB; lazy 3D and charts	329.1 kB / 330.0 kB gzipped across 12 eager chunks; lazy boundaries verified	Passed
Reliability (Static Code & Fault Analysis)	SonarQube Community Build 26.7.0.124771	Quality Gate PASS; 0 Bugs; Rating A	0 Bugs, 0 Vulnerabilities, 0 Security Hotspots across 224 files; Reliability A, Security A, Duplication 0.6%	Passed

ISO/IEC 25010 Characteristic	Automated Instrument & Version	Enforced CI / Quality Gate	Measured Empirical Result	Status
Reliability (Supply-Chain & Container Security)	Trivy v0.36.0, pip-audit, npm audit, Gitleaks v2	0 Critical or High CVEs; 0 leaked secrets	0 vulnerabilities across locked Python/Node dependencies and container images; 7 / 7 CI checks green	Passed
Maintainability (Backend & Frontend Linters)	Pylint v4.1.3 & ESLint v9.39.5	Pylint = 10.00/10; ESLint = 0 errors or warnings	Backend Pylint: 10.00 / 10 (0 messages); Frontend ESLint: 0 errors, 0 warnings; Sonar Maintainability A	Passed

Figure 11

ISO/IEC 25010 Automated Quality Gate Results and Apache JMeter Latency Profiles



Under Functional Suitability, the backend PyTest suite executed 2,343 unit and integration tests (with 3 opt-in disposable-database RLS tests skipped by default, 2,346 collected in total) with zero failures and 92.08% statement coverage across 6,607 statements (exceeding the 85.0% CI gate), while the frontend unit test suite passed all 271 tests across 15 suites at 96.76% line coverage, 87.45% branch coverage, and 96.45% function coverage. End-to-end browser verification via Playwright v1.61.1 and axe-core v4.12.1 passed all 34 browser tests (16 critical user journeys and 18 surface accessibility suites) and recorded zero serious, critical, or advisory accessibility violations across 90 surface-theme-viewport scans.

Under Performance Efficiency, Apache JMeter v5.6.3 stress testing across 900 requests with 20 concurrent virtual users achieved a 100% success rate (900 of 900 HTTP 200, 0.00% errors) with an arithmetic mean response time of 83.78 ms, a median of 72.00 ms, a 95th percentile (p95) of 204.05 ms, and a throughput of 10.12 requests per second.

The rate-limiting test plan (60 requests) enforced the exact 30-request window (30 HTTP 200 followed by 30 HTTP 429) at a mean latency of 5.17 ms (p95 = 6.00 ms). On the live Gemini release smoke test, embedding generation averaged 514.99 ms, verdict generation 806.15 ms, and grounded chat synthesis 1,521.93 ms (multi-iteration median 1,196 ms). Under concurrent RAG chat load, grounded synthesis completed in a median of 8,071 ms (p95 = 14,134 ms) at 5 concurrent users and degraded gracefully at 10–20 concurrent users via the 60-second capacity cooldown notice (1–18 ms response time, 0% HTTP 5xx server failures).

Under Reliability and Maintainability, continuous static analysis via SonarQube Community Build 26.7.0.124771 across 102 Python and 122 JavaScript source files passed the Quality Gate with 0 bugs, 0 vulnerabilities, 0 security hotspots, 0.6% code duplication, and A ratings across Reliability, Security, and Maintainability. Container and dependency audits (Trivy v0.36.0, pip-audit, npm audit, and Gitleaks) confirmed zero open vulnerabilities across both production container images and dependency lockfiles, while Pylint v4.1.3 achieved a perfect 10.00/10 score and ESLint v9.39.5 reported zero errors and zero warnings.

4.4.1 Computational Observations

Two operational observations highlight the architectural behavior of the platform. First, per-query trace inspection confirmed that the two-stage hybrid retriever consistently locates relevant thesis passages at cosine similarities of 0.67–0.74 across both five-chapter and IMRAD manuscripts, while the structural citation validator acts as a deterministic epistemic gate that withholds any response whose substantive claims lack valid in-range [n] markers. Second, separating the five-year active recency window from long-term

archive storage allows students and faculty advisers to focus literature reviews and novelty checks on the most recent five years of CCSICT scholarship while preserving complete institutional memory across historical cohorts.

CHAPTER V

SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

This chapter synthesizes the empirical findings of the study in accordance with the general and specific objectives established in Chapter 1, formulates evidence-grounded conclusions on the performance and internal software quality of the Centralized AI-Powered Thesis Library System (IskAI), and presents actionable recommendations for institutional deployment and future research.

5.1 Summary of Findings

This study engineered and evaluated a closed-domain Retrieval-Augmented Generation (RAG) architecture and web-based thesis library platform for the College of Computing Studies, Information and Communication Technology (CCSICT) at Isabela State University – Echague Campus. Organized by the four specific objectives of the study, the principal findings are as follows:

1. Localized RAG + LLM Knowledge Retrieval Architecture (Objective 1): A localized retrieval-to-generation pipeline was conceptualized, implemented, and locked under a cryptographic release manifest. The digitization layer combines PyMuPDF v1.28.0 text extraction with a Tesseract OCR v2.10.0 fallback, a 15% non-alphanumeric noise filter, deterministic Republic Act No. 10173 PII redaction, and dual structural support for both traditional five-chapter manuscripts (Chapters 1–5 and Roman numerals I–V) and IMRAD manuscripts (Introduction, Materials and Methods, Results and Discussion). The front-matter extraction module isolates seven standardized bibliographic and committee attributes—Thesis Title, Authors, Academic Year, Degree Program and Specialization,

Faculty Thesis Adviser, Panel Chairman, and Panel Members—together with the verbatim Abstract directly from the title page and Approval Sheet. Cleaned manuscripts are segmented by a section-aware RecursiveCharacterTextSplitter into 800-token chunks with a 100-token overlap, embedded into 768-dimensional vectors via Gemini Embedding in Supabase pgvector, and retrieved through a two-stage pipeline (0.30 cosine similarity floor, candidate pool of 15, deterministic hybrid rerank weighting 0.75 cosine similarity and 0.25 section and lexical overlap, per-paper cap of 3, top-k = 5 selection, LongContextReorder, and structural [n] citation validation).

2. Comparative Factual Accuracy and Hallucination Mitigation (Objective 2): In the formal 40-query evaluation (Run ID 5e8fb7f21db6, validated by a three-member CCSICT faculty panel and scored with Ragas v0.4.3 using Gemini 3.8 Flash), the proposed RAG + LLM pathway increased pooled Answer Correctness from 0.2169 (Baseline LLM) to 0.3024, yielding a statistically significant mean paired gain of +0.0855 (+39.4% relative gain, 95% CI [+0.0487, +0.1223], Shapiro-Wilk $W = 0.9594$, $p = 0.1595$; paired $t(39) = 4.6990$, $p = 3.22 \times 10^{-5}$, Cohen's $d = 0.7430$). Stratified analysis demonstrated statistically significant hallucination mitigation across all out-of-scope and unindexed query strata (Unstudied Computing Topics $\Delta = +0.0866$, $p = 0.0009$, $d = 0.9858$; Unindexed Track Scope $\Delta = +0.1610$, $p = 0.0419$, $d = 1.7095$; Out-of-Scope Negative Controls $\Delta = +0.1139$, $p = 0.0122$, $d = 5.1819$) and a +26.1% relative gain in Answer Correctness on Covered Topics (0.2302 to 0.2903, $\Delta = +0.0601$). Across the 32 queries where the RAG pipeline emitted a substantive cited answer, mean Faithfulness reached 0.7793 (pooled 0.6634) and Context Precision reached 0.2221.

3. Full-Stack System Integration and Institutional Workflow Support (Objective 3): The validated model was integrated into a production web application (React v19.2.8, Vite v8.3.6, Tailwind CSS v4.3.3, FastAPI v0.139.2, and Supabase) encompassing five operational subsystems: (a) Single and Batch ingestion for up to 20 concurrent manuscripts with Approval-Sheet autofill of Title, Authors, Year, Adviser, Panel Chair, Panel Members, and Abstract, inline diagnostic notes and required-field validation, and an In-Document versus AI-Synthesized Extended Abstract (400–550 words) toggle; (b) an indirect Thesis Archive and Grounded RAG Chat Assistant with 12 deterministic fast paths, Filipino and Ilocano cross-lingual query translation, complete committee context headers, [n] citation cards, interactive research tools (prompt branch editing, text-to-speech audio playback, research note export, and provenance inspection), and real-time 85% duplication warnings; (c) a 5-year recency filter (All theses versus Last 5 years) across Archive, Chat, and Novelty Check paired with an administrative 5-Year Storage and Retention governance console; and (d) Single, Batch, and 4-Section Research Capsule Novelty Screening with deterministic and semantic PDF, Microsoft Word, and plain-text proposal extraction, single-paper concentration grading (< 25% Clear, 25–69.9% Review Suggested, ≥ 70% High Overlap), post-ingest incremental rescanning, and client-side export of an official Microsoft Word Novelty Check Report bearing signatory blocks for the Thesis Adviser and the Dean of CCSICT (Christine Charmaine G. San Jose, DIT).

4. ISO/IEC 25010 Internal Software Quality Evaluation (Objective 4): Automated evaluation across the four internal quality characteristics of ISO/IEC 25010 passed every enforced gate: (a) Functional Suitability achieved a 100% pass rate across 2,343 executed backend PyTest unit and integration tests (3 opt-in disposable RLS tests skipped; 92.08%

coverage across 6,607 statements vs. 85.0% gate), 271 frontend unit tests (96.76% line, 87.45% branch, and 96.45% function coverage), and 34 Playwright end-to-end and accessibility browser tests with zero WCAG 2.2 AA accessibility violations across 90 axe-core scans; (b) Performance Efficiency recorded 900 of 900 successful responses (0.00% error) under 20-user Apache JMeter load with a mean latency of 83.78 ms, median of 72.00 ms, p95 of 204.05 ms, rate-limiter enforcement at p95 = 6.00 ms, live Gemini smoke median latency of 1,196 ms, and an eager gzipped frontend bundle of 329.1 kB against the 330.0 kB budget; (c) Reliability passed the SonarQube Community Build 26.7.0.124771 Quality Gate across 224 source files with 0 bugs, 0 vulnerabilities, 0 security hotspots, A ratings across Reliability, Security, and Maintainability, 0.6% duplication, and zero open CVEs in Trivy, pip-audit, npm audit, and Gitleaks; and (d) Maintainability achieved a 10.00/10 Pylint v4.1.3 score and zero ESLint v9.39.5 errors or warnings.

5.2 Conclusions

Based on the empirical measurements and automated verification artifacts presented in Chapter 4, the following conclusions are drawn:

1. On the Knowledge Retrieval Architecture (Objective 1): Combining layout-aware digitization, dual five-chapter and IMRAD section-aware chunking (800 tokens with 100-token overlap), 768-dimensional vector indexing, two-stage hybrid lexical-cosine reranking, LongContextReorder, and approval-sheet metadata injection resolves the semantic gap of keyword-only catalog systems while preserving complete institutional attribution (Title, Authors, Adviser, Panel Chair, Panel Members, Program, Year, and Page and Section) without exposing full-text thesis PDFs.

2. On Factual Accuracy and Hallucination Mitigation (Objective 2): Constraining the Large Language Model to localized Supabase pgvector evidence significantly outperforms an unaugmented Baseline LLM on pooled Answer Correctness (+0.0855, $p = 3.22 \times 10^{-5}$) and statistically eliminates ungrounded fabrication across out-of-scope queries ($p = 0.0009$ to $p = 0.0419$), achieving a mean Faithfulness of 0.7793 across substantive cited responses and a +26.1% relative improvement on covered institutional queries.

3. On System Integration and Institutional Utility (Objective 3): Integrating automated Approval-Sheet committee autofill, single and batch worker ingestion, five-year recency scoping, pre-manuscript four-section Research Capsule screening at the 85% cosine similarity threshold, and official Microsoft Word endorsement report generation into a unified web platform streamlines literature review discovery and title-defense novelty validation for CCSICT students, faculty advisers, and administrators.

4. On Software Product Quality (Objective 4): The developed platform satisfies the internal quality requirements of ISO/IEC 25010 across Functional Suitability, Performance Efficiency, Reliability, and Maintainability, as evidenced by a 100% automated test pass rate (2,614 executed backend and frontend unit tests across 2,617 collected, plus 34 E2E and accessibility browser tests), sub-205 ms p95 API latency under concurrent load, zero static analysis bugs or security vulnerabilities, and a 10.00/10 backend maintainability score.

5.3 Recommendations

In light of the findings of this study, the researchers recommend the following actions for institutional deployment and future research:

1. Continuous Semesterly Ingestion Across All CCSICT Programs: Utilize the concurrent Batch Upload subsystem and dual five-chapter and IMRAD ingestion pipeline to continuously index newly approved undergraduate and faculty manuscripts at the conclusion of each academic semester across all CCSICT degree programs and specializations (BSCS, BSIT, BSIS, BLIS, and BSDSA).

2. Institutional Adoption of Capsule Screening and Official Endorsement Reports: Adopt the Research Capsule Novelty Screening mode and the downloadable Microsoft Word Novelty Check Report (bearing the endorsement signature blocks for the Thesis Adviser and the CCSICT Dean) as a standard pre-defense requirement during capstone and thesis title proposals, using the Last 5 years recency toggle to enforce the five-year active research recency policy while retaining older theses in the historical archive.

3. Citation-Repair Token Headroom Optimization: Configure production deployments with explicit reasoning-effort bounds and raised output token ceilings on citation-repair calls so that multi-paragraph syntheses over dense technical manuscripts complete structural citation verification in a single pass.

4. Future Research Extensions: Future researchers should (a) evaluate cross-encoder reranking stages to further elevate Context Precision, (b) benchmark Ragas scores using an independent cross-family judge model anchored against a human-scored subsample, (c) provision dedicated paid-tier LLM API quotas for high-concurrency department-wide defense periods, and (d) integrate multimodal vision-language parsing to index complex Entity-Relationship Diagrams (ERDs) and architectural figures alongside manuscript prose.

References

Arivazhagan, M. G., Liu, L., Qi, P., Chen, X., Wang, W. Y., & Huang, Z. (2023). Hybrid hierarchical retrieval for open-domain question answering. *Findings of the Association for Computational Linguistics: ACL 2023*, 10680–10689. https://doi.org/10.18653/v1/2023.findings-acl.679

Armilla, R., & Abangan, M. L. (2025). The difficulties and strategies in digitizing library resources. *Psychology and Education: A Multidisciplinary Journal, 31*(9), 933–962. https://doi.org/10.5281/zenodo.14828861

Aytar, A. Y., Kaya, K., & Kılıç, K. (2025). A synergistic multi-stage RAG architecture for boosting context relevance in data science literature. *Natural Language Processing Journal, 10*, 100179. https://doi.org/10.1016/j.nlp.2025.100179

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*. https://doi.org/10.1145/3644815.3644945

Besrour, I., He, J., Schreieder, T., & Färber, M. (2025). SQuAI: Scientific question-answering with multi-agent retrieval-augmented generation. *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM 2025)*. https://doi.org/10.1145/3746252.3761471

Bevara, R. V. K., Lund, B. D., Mannuru, N. R., Karedla, S. P., Mohammed, Y., Kolapudi, S. T., & Mannuru, A. (2025). Prospects of retrieval augmented generation (RAG) for academic library search and retrieval. *Information Technology and Libraries, 44*(2). https://doi.org/10.5860/ital.v44i2.17361

Bodea, A.-E., Meisenbacher, S., Klymenko, A., & Matthes, F. (2026). SoK: Privacy risks and mitigations in retrieval-augmented generation systems. *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML 2026)*. https://arxiv.org/abs/2601.03979

Brown, A., Roman, M., & Devereux, B. (2025). A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges. *Big Data and Cognitive Computing, 9*(12), 320. https://doi.org/10.3390/bdcc9120320

Custer, S., & Sethi, T. (2017). *Avoiding data graveyards: Insights from data producers & users in three countries*. AidData, College of William & Mary. https://www.aiddata.org/publications/avoiding-data-graveyards-insights-from-data-producers-users-in-three-countries

Doliente, C. J. O., Tual, D.-X., Paglinawan, Z. B., Naparan, J. A., & Facunla, H. L. (2023). Exploring students' utilization of Online Public Access Catalog in the learning common. *International Journal of Science and Applied Information Technology, 12*(4), 29–37. https://doi.org/10.30534/ijsait/2023/011242023

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. *Proceedings of the 18th Conference*

of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 150–165. https://arxiv.org/abs/2309.15217

Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 6465–6488. https://arxiv.org/abs/2305.14627

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. *arXiv preprint*, arXiv:2312.10997. https://doi.org/10.48550/arXiv.2312.10997

Gokdemir, O., Siebenschuh, C., Brace, A., Wells, A., Hsu, B., Hippe, K., Setty, P., Ajith, A., Pauloski, J. G., Sastry, V., Foreman, S., Zheng, H., Ma, H., Kale, B., Chia, N., Gibbs, T., Papka, M., Brettin, T., Alexander, F., . . . Ramanathan, A. (2025). HiPerRAG: High-performance retrieval augmented generation for scientific insights. *Proceedings of the Platform for Advanced Scientific Computing Conference (PASC 2025)*. https://doi.org/10.1145/3732775.3733586

Google. (2024). *Gemini API terms of service*. Google for Developers. https://ai.google.dev/gemini-api/terms

Khaliq, A., Abbasi, S. B., Ilyas, A., Shaikh, S. M., & Ali, S. A. (2024). Optimizing academic queries with retrieval-augmented large language models. *Migration Letters, 21*(S10), 1274–1283. https://doi.org/10.59670/ml.v21is10.11858

Kovanen, S. (2025). *Hallucination mitigation for faithful retrieval-augmented generation—Investigating layer-contrastive and attention-based decoding methods* [Master's thesis, Aalto University]. Aaltodoc. https://aaltodoc.aalto.fi/items/fff2741f-4076-49c1-9efa-f8f3ff2968c8

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kutuzov, A., Lewis, M., Yih, W., Rocktäschel, T., & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474. https://arxiv.org/abs/2005.11401

Li, Z., Wang, Z., Wang, W., Hung, K., Xie, H., & Wang, F. L. (2025). Retrieval-augmented generation for educational application: A systematic survey. *Computers & Education: Artificial Intelligence, 8*, 100417. https://doi.org/10.1016/j.caeai.2025.100417

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Mezhlumyan, A. (2024). *Building a retrieval-augmented generation (RAG) system for academic papers* [Capstone project, American University of Armenia]. AUA Repository. https://cse.aua.am/wp-content/uploads/2024/05/Building-a-Retrieval-Augmented-Generation-RAG-System-for-Academic-Papers.pdf

Open Web Application Security Project (OWASP). (2025). *OWASP Top 10 for Large Language Model applications*. OWASP Foundation. https://owasp.org/www-project-top-10-for-large-language-model-applications/

Oreški, D., & Vlahek, D. (2024). Retrieval augmented generation in large language models: Development of AI chatbot for student support. *Proceedings of the 15th International Conference on e-Learning (eLearning 2024)*. CEUR Workshop Proceedings, Vol. 3938. https://ceur-ws.org/Vol-3938/Paper_2.pdf

Ru, D., Qiu, L., Hu, X., Zhang, T., Shi, P., Chang, S., Jiayang, C., Wang, C., Sun, S., Li, H., Zhang, Z., Wang, B., Jiang, J., He, T., Wang, Z., Liu, P., Zhang, Y., & Zhang, Z. (2024). RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. *arXiv preprint*, arXiv:2408.08067. https://arxiv.org/abs/2408.08067

Sambrano, L. C., Reyes, J. R. B., Fabro, R. B. B., Colar, C. K. F., & Medrano, V. B. (2025). The influence of artificial intelligence (AI) on library patronage among college students. *International Journal of Research and Innovation in Social Science, 9*(7). https://doi.org/10.47772/IJRISS.2025.90700010

Santra, P. P., Kashyap, R., Mahata, A., & Sk, M. A. (2025). Leveraging retrieval-augmented generation in local library systems: The BiblioGPT prototype. *14th International CALIBER 2025*. https://ir.inflibnet.ac.in/items/f2bcd28e-ae2d-4dbd-8ed7-c7b987b3c30c

Supabase. (2026). *pgvector: Embeddings and vector similarity search*. Supabase Technical Documentation. https://supabase.com/docs/guides/database/extensions/pgvector

Supreme Court of the Philippines. (2024). *Chief Justice Gesmundo: Judiciary E-Library to use AI technology to improve legal research*. Public Information Office. https://sc.judiciary.gov.ph/chief-justice-gesmundo-judiciary-e-library-to-use-ai-technology-to-improve-legal-research/

Vercruysse, A., Rojas Melendez, J. A., & Colpaert, P. (2024). Linking application and semantic data with RDF Lens. *Proceedings of the 20th International Conference on Semantic Systems (SEMANTiCS 2024)*. CEUR Workshop Proceedings, Vol. 3759. https://ceur-ws.org/Vol-3759/paper13.pdf

Wong, L., Ali, A., Xiong, R., Shen, S. Z., Kim, Y., & Agrawal, M. (2025). Retrieval-augmented systems can be dangerous medical communicators. *arXiv preprint*, arXiv:2502.14898. https://arxiv.org/abs/2502.14898

Xuan, W., & Shaw, C. (2025). Reimagining library services in the age of AI: A case study from a Canadian academic library. *Proceedings of the IATUL Conferences*, Paper 4. https://docs.lib.purdue.edu/iatul/2025/breakout/4/